Structural Bioinformatics

Contents

Basics

Basics I

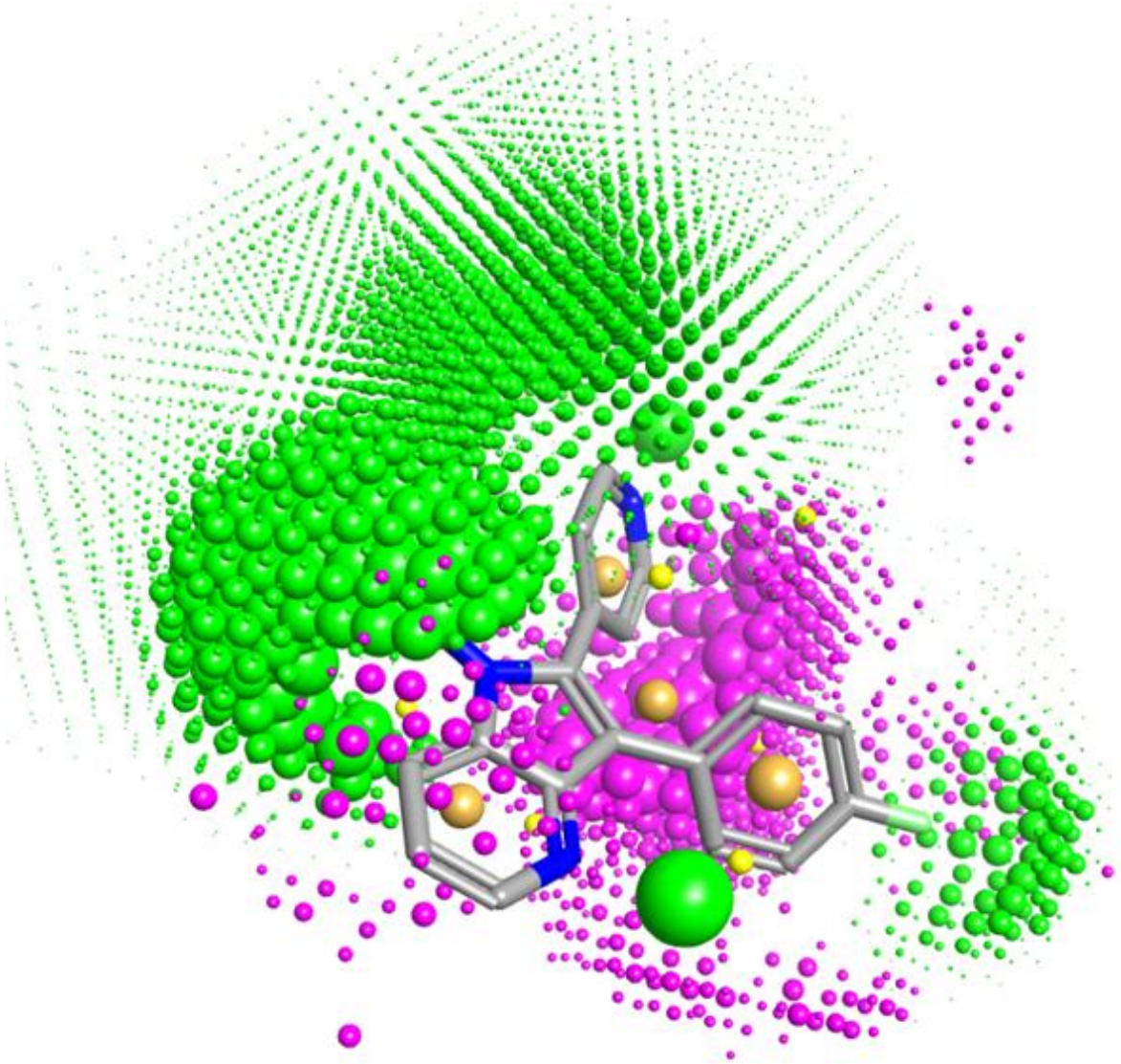
- Is another field of bioinformatics
- Works with three-dimensional structure data of macromolecules and small organic molecules like ATP
- Mainly in association with Protein structures but in the last years the field was expanded into RNA/DNA structures as well even though we are still in the development stages here
- Hosts many sub fields that deal with very specific aspects

Basics II

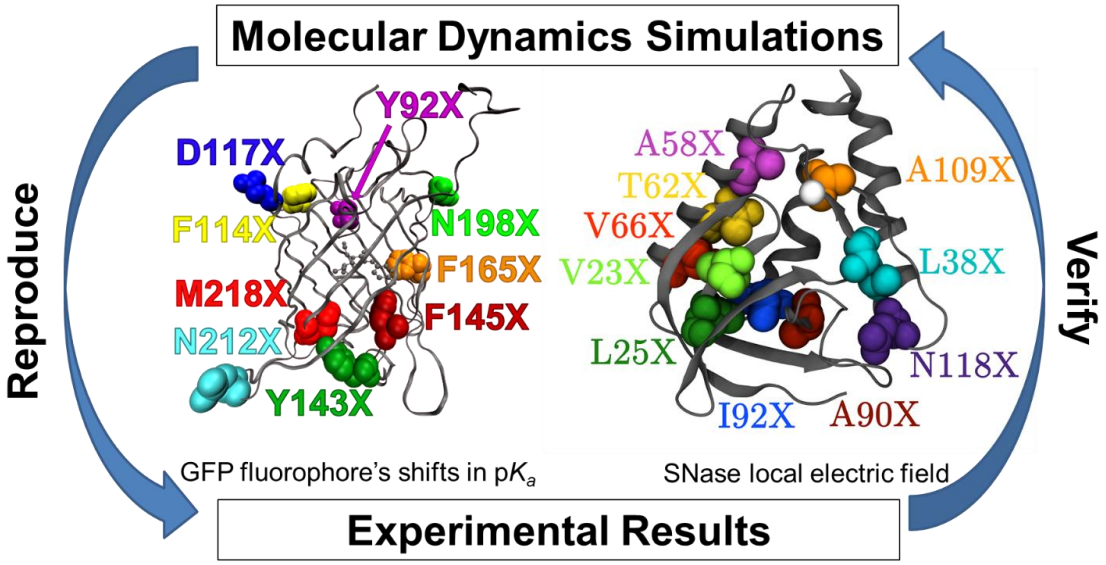
- Some of the main sub fields are
  - Drug-Design with 3D-QSAR
  - Analysis of protein dynamics with Molecular simulations
  - Creation of new protein structures *in silico*, with the help of algorithms using Homology Modelling, Threading or ab initio Modelling
  - Research of Protein-Protein or Protein-Ligand interactions with programs like Docker
  - Assessment of structures

Basics III – 3D-QSAR

- 3D - Quantitative structure–activity relationship
- Analyzing binding pockets regarding bio-physical / biochemical properties
- Finding small organic molecules that fit these criteria
- Calculate potential binding constants to determine whether they are better or not than already exisiting ligands

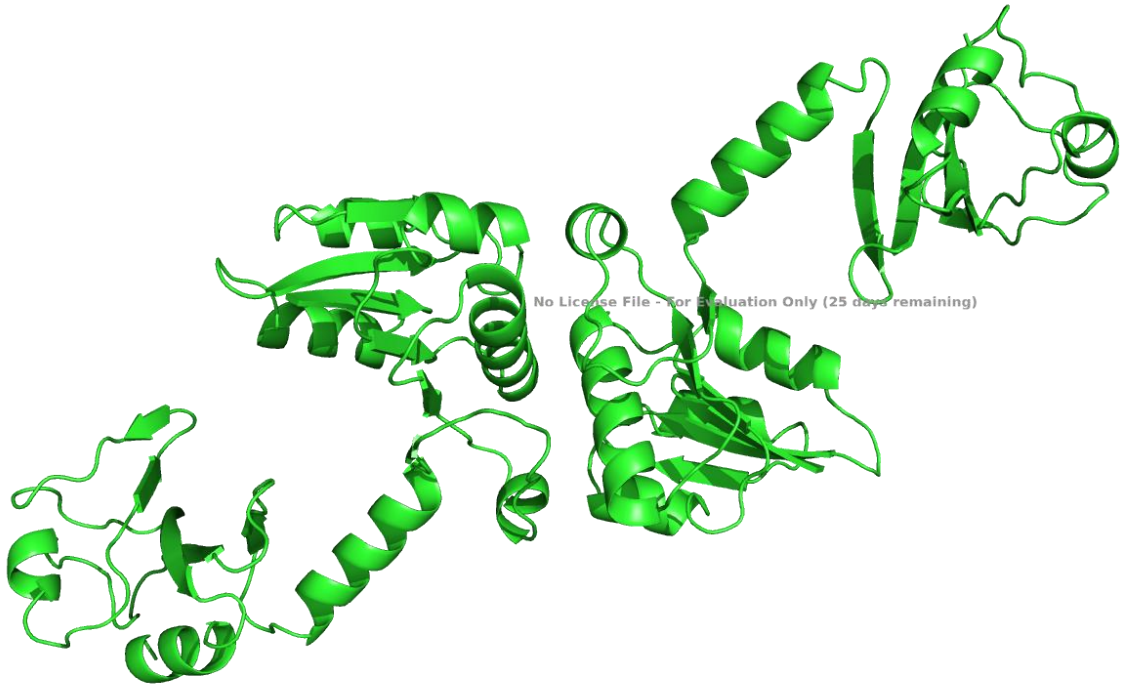


Basics IV – Molecular Dynamics (MD) simulations [1]



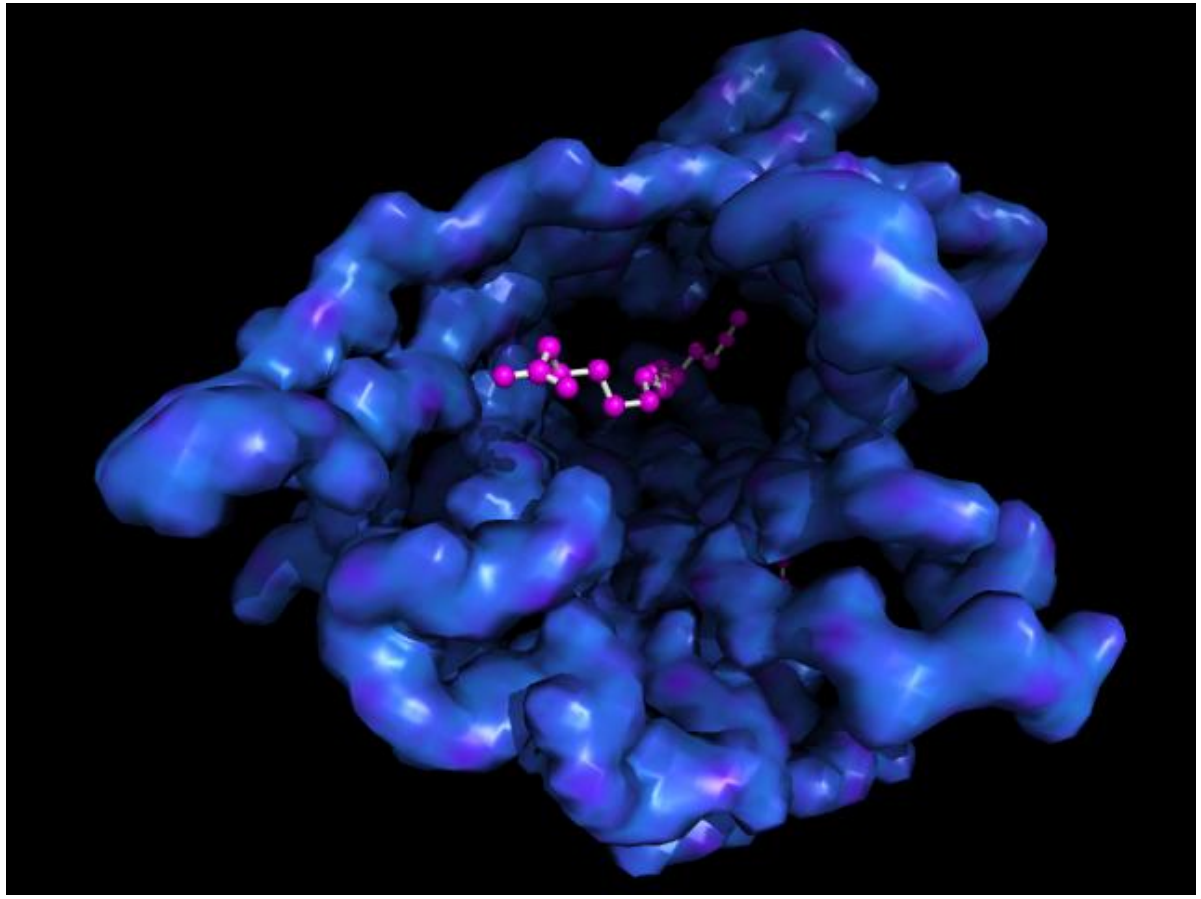
- Using energy functions and force field calculations to simulate the motion of macromolecular structures
- Current simulations are able to simulate the contents of parts of a cell, including all proteins present in there

Basics V – Homology Modelling



- We do not know the three-dimensional structure for all known proteins
- To be able to analyze these proteins on a structural level we can make use of known structures to generate models for proteins we don‘t know the structure of

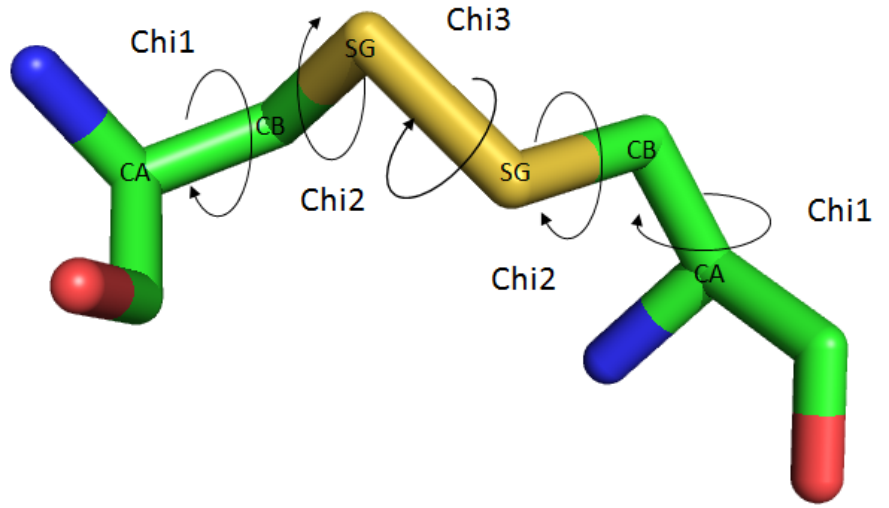
Basics VI – Docker [2]

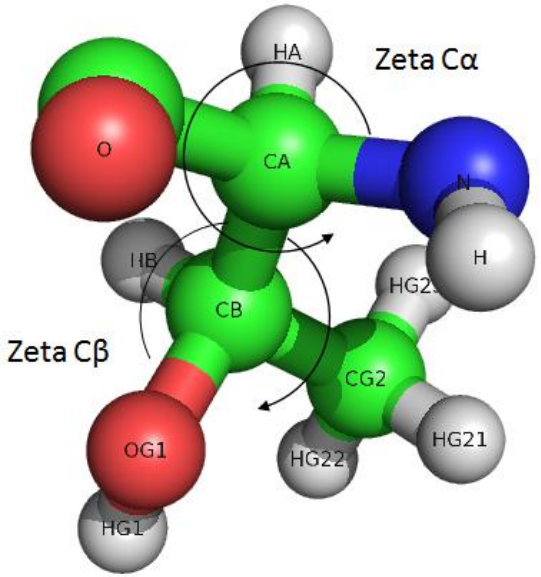


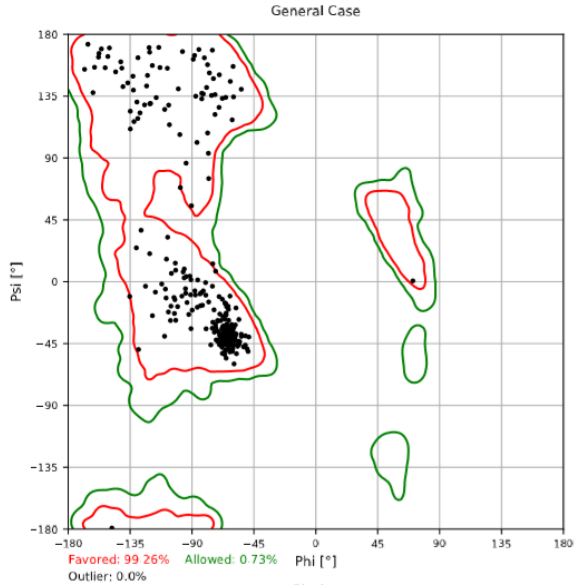
- We can analyze the binding properties of small organic molecules / proteins / nucleic acids on other proteins using docker
- The program calculates potential binding configurations by fitting for example a small organic molecule into a binding pocket of a protein structure

Basics VII – Structure Assessment

- Three-dimensional protein structures, especially older ones are relatively error-prone
- We need ways to analyze their quality by making use of simple geometric aspects
- We can also use nearings to determine the overall energy state of structures

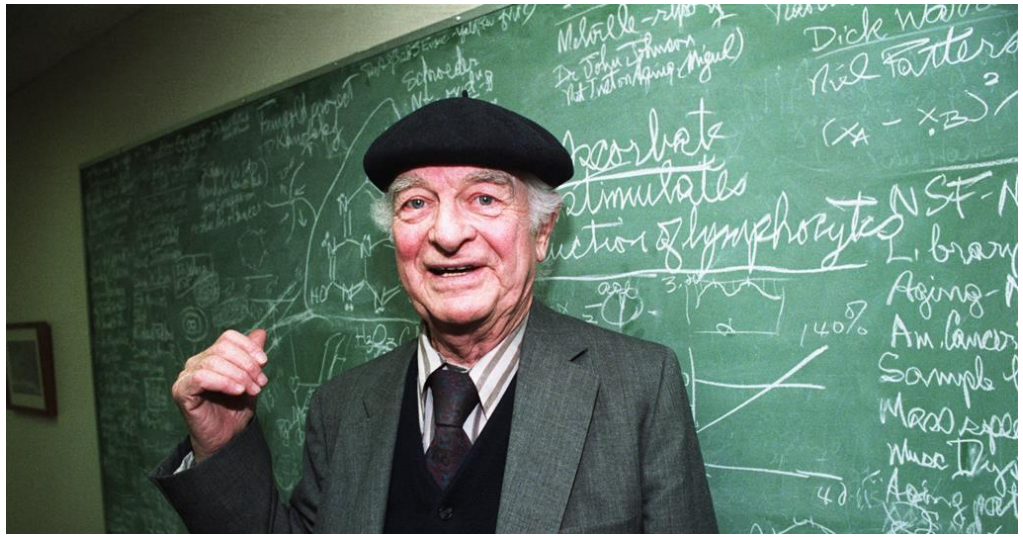


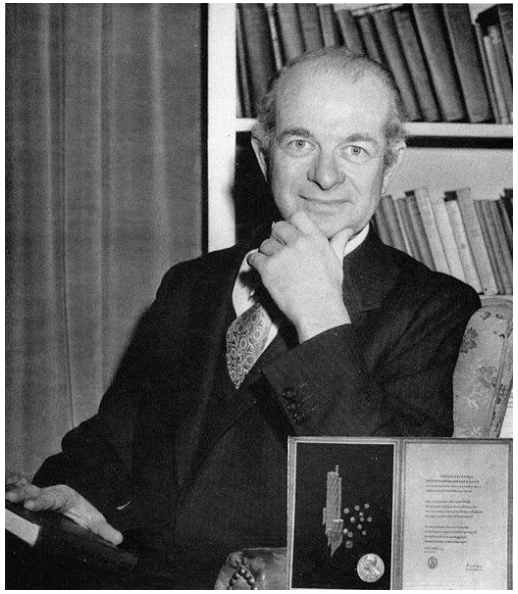


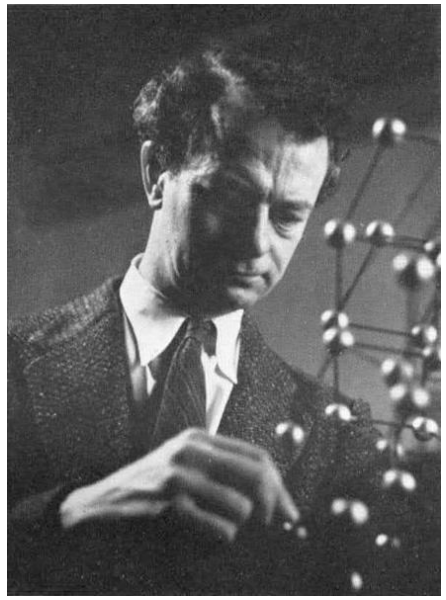


Linus Pauling

Linus Pauling I [3]







Linus Pauling II [3]

- Won two unchared Nobel Prizes, in Chemistry 1954 and for Peace in 1962
- Postulated the structure of α-Helices [4] and β-Sheets [5,6] in 1951
- Even Albert Einstein said about him: „Now that is a genius“ after visiting a lecture of him
- His stance of the usage of vitamines is seen rather critical, however some of his statements were later recognized

Linus Pauling III [7]

- The pivotal moment came in the early spring of 1948, when Pauling caught a cold and went to bed
- Being bored, he drew a polypeptide chain of roughly correct dimensions on a strip of paper and folded it into a helix, being careful to maintain the planar peptide bonds
- After a few attempts, he produced a model with physically plausible hydrogen bonds
- However his model did not fit the fotos available of helices
- He found out that the turns of a helix repeat every 5.4 Å, however the fotos indicated a distance of 5.1 Å [8]

Linus Pauling IV

- Ever the perfectionist Linus Pauling did decide to not publish his founding on the alpha-Helix until a more detailed publication showed that the real distance between the helix repeats was in deed 5.4 Å, just like he predicted [8]
- If he would have been able to see the fotos of DNA made by Rosalind Franklin he probably had beaten Watson and Crick in the discovery of the double helical structure of DNA as well [8]

Linus Pauling V

- Not only did Pauling play an essential role in many discoveries relating protein structures
- He was also one of the first scientists to lay the focus in research on structural explanations be it for deseases or natural phenomena
- He was for example able to deduct what problems arose in patients with sickle cell disease on a structural basis just by talking to scientist / doctor who described the symptoms and observations of the blood cells
- Therefore he could be seen as the predecessor of the current structural biochemists

Protein Structures

Protein Structures I

- The three dimensional structure of a protein is a result of different interactions in the protein:
  - Hydrogen Bonds
  - Van-der-Waals Bonds (non polar interactions)
  - Disulfide Bonds
  - Salt bridges
  - Dipol interactions
  - Aromatic interactions
  - Interactions between Cations and aromatic ring systems
  - …

Protein Structures II [9]

- A protein tries to assume a structure that has the least possible energy while maintaining several other facts that can increase the energy:
  - Creating a hydrophile surface
  - Creating a hydrophobic core
  - Having a functional active centre
  - Be able to bind molecules like ATP and GTP
  - Catalyze essential reactions in the cell
- The folding process is dependent on many factors like temperature, ph, solvent and others

Protein Structures III – Structure Planes I

- A protein consists of amino acids and each protein has a distinct three-dimensional structure
- Each protein has at least three, in some cases even four, structure planes:
  - Primary structure
  - Secondary structure
  - Tertiary structure
  - Quartery structure

Protein Structures IV – Primary Structure [9]

- The primary structure of proteins is the amino acid sequence that makes up the protein
- This structure has no real three-dimensional equivalent, nevertheless the amino acid sequence not only determines the native structure of a protein it also specifies the pathway to reach this native structure

- MDQSNRYAMLTLVEENLVMDGNHLLVAYKLKPAVCY

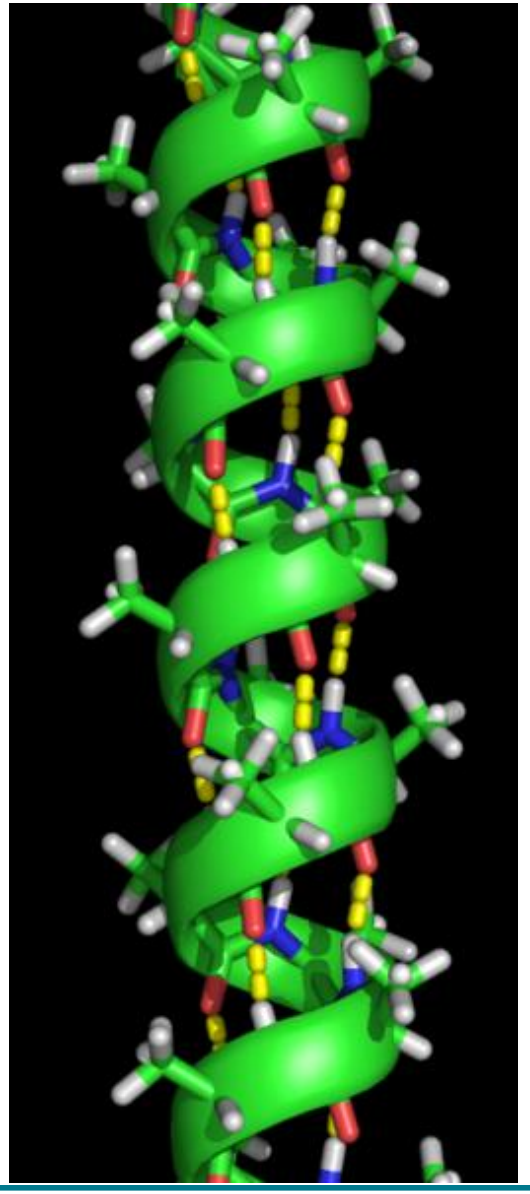
Protein Structures V – Secondary structure

- The secondary structure of a protein consists of special elements that are part of the structure
- The two main players are:
  - α-Helix
    - There are other types as well, but in most cases they are just special cases of an α-Helix
  - β-Sheets
    - parallel or anti-parallel
- There is a third type called Turns or Loops that is not as defined as the first two

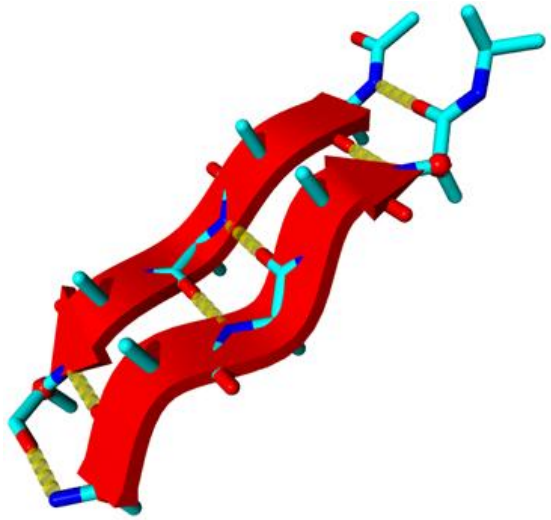
Protein Structures VI – α-Helix [10], β-Sheets [11] and Turns

- On the right side you can see cartoon representations of an α-Helix (A), an anti-parallel β-Sheet (B, the red arrows) and a Turn between a Helix and a Strand (C, in green)
- In yellow essential hydrogen bonds between the amino acids are shown

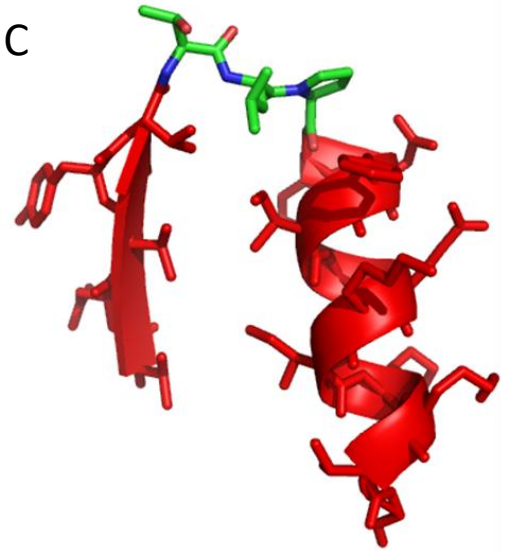
A



B



C



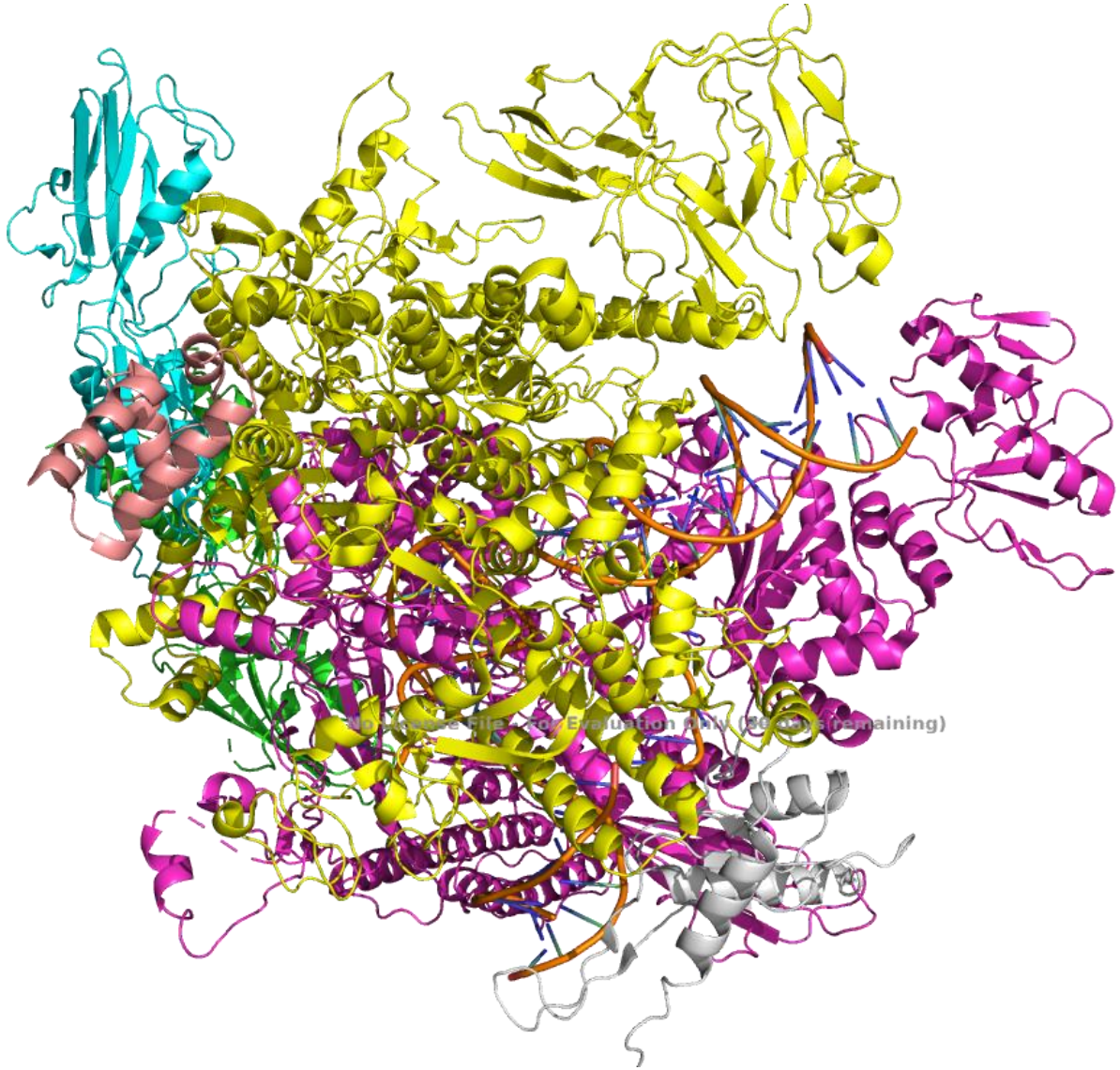
Protein Structures VII – Tertiary Structure (1kim)

- The tertiary structure of a protein is the complete three-dimensional structure of one amino acid chain, that means one macromolecule
- It includes all α-Helices, β-Sheets, Turns and unstructured regions of the protein
- Represents one of the functional low-energy states of the protein

Protein Structures VIII – Quartery Structure

- The quartery structure of a protein is the assembly of multiple proteins in one bigger complex
- Can be homogenous
  - Two or more proteins with the same structure (Dimer, Trimer, etc.)
- Or heterogenous:
  - Complex of different proteins to fulfill a complex task, e.g. Replication complex

Protein Structures IX – Quartery Structure (7PY8)



PyMol – Structure Visualization

PyMol I [12]

- One of the most common tools to look at Protein structures is PyMol and you can download a free 30-Days test version under:
  - https://pymol.org/2/
- Open source but proprietary molecular visualization system created by Warren Lyford DeLano
- It was commercialized initially by DeLano Scientific LLC, which was a private software company dedicated to creating useful tools that become universally accessible to scientific and educational communities
- It is currently commercialized by Schrödinger, Inc.

PyMol II [12]

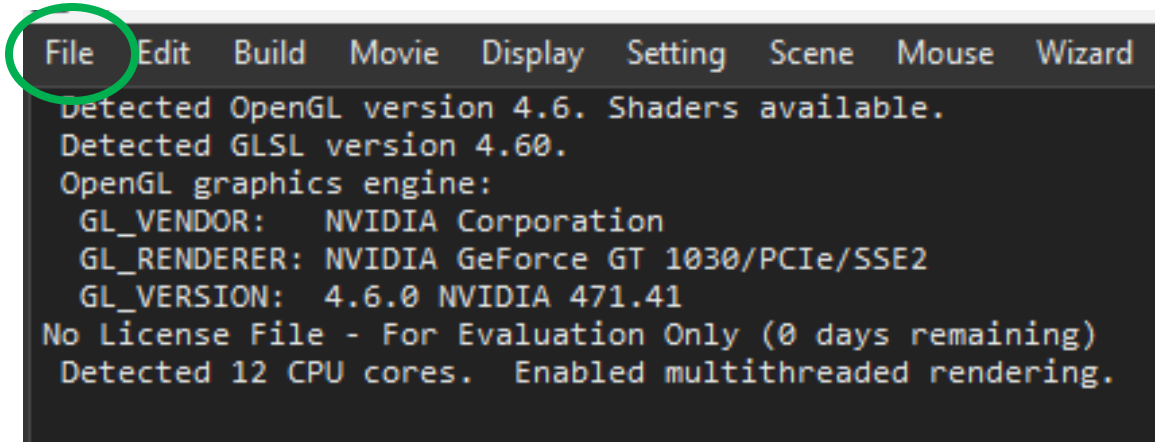
- PyMol can produce high-quality 3D images of small molecules and biological macromolecules, such as proteins or nucleic acids
- According to the original author, by 2009, almost a quarter of all published images of 3D protein structures in the scientific literature were made using PyMol
- PyMol is one of the few mostly open-source model visualization tools available for use in structural biology
- The Py part of the software's name refers to the program having been written in the programming language Python

PyMol III – Uses and limitations

- PyMol is a VISUALIZATION tool, that means it was mainly developed to look at our structures (or parts of them) and change how they are shown
- However over the course of time more and more editing functions, as well as measurement functions were included in it
- Nowadays PyMol is a very complex and mighty program, especially when used in combination with Python

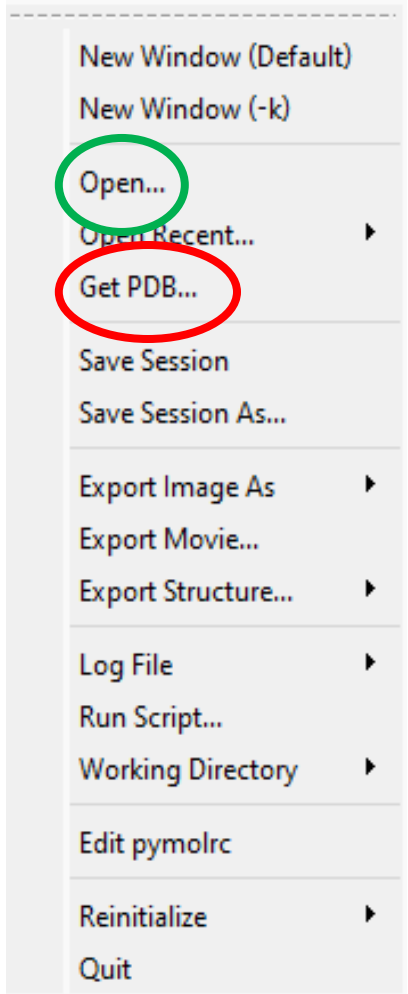
PyMol IV – A Session

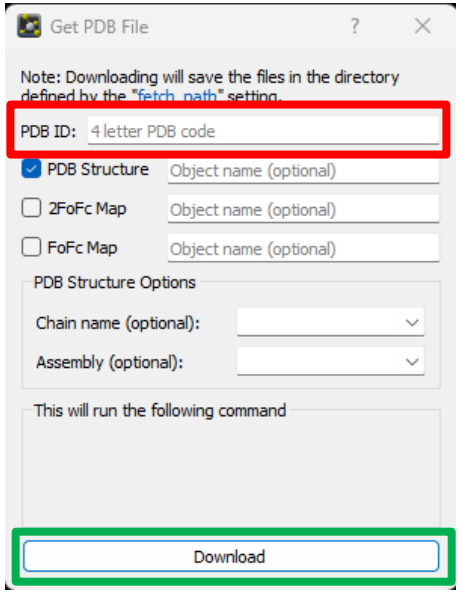
- When we work with PyMol we are working in a so-called PyMol session
- In these sessions we are able to look at multiple different structures at once, to compare them with each other for example
- When you start PyMol the session does not contain any structures
- So we need to load these structures manually with the help of the file menu at the top of PyMol (image below, green circle)



PyMol V – Loading Structures

- In the file menu (right) we can either open a pdb (Open…, green circle) or we can use the option Get PDB… (red circle) to download a structure from the RCSB-PDB into our session
- If we do the later, a Pop-Up window (below) opens and we can type in the PDB-Code into the entry field (red) and download it (green)





PyMol VI – Structure sidebar

- Once you loaded structures into your PyMol Session they appear on the right side of the window in Form of a sidebar (top)
- By clicking on the name of the structures you can toggle whether they are active (grey background, 1bl8 and 1kim at the bottom) or inactive (black background, 3fqk, 4p5j and 7py8 at the bottom)

- Only active structures can be seen in the current session, but the inactive ones are not deleted but hidden
- Each structure has five menus called Action (A), Show (S), Hide (H), Label (L) and Color (C)

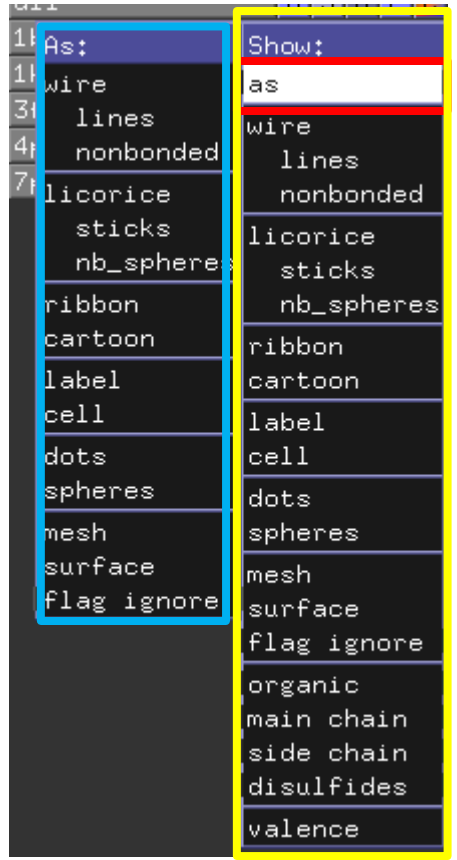
PyMol VII – Representations I

- PyMol allows us to visualize our structures in form of many different representations
- On the right side you can see a list of some of the most commonly used ones
- All of them are available in the “Show” Menu of your structures you have available in your current PyMol session

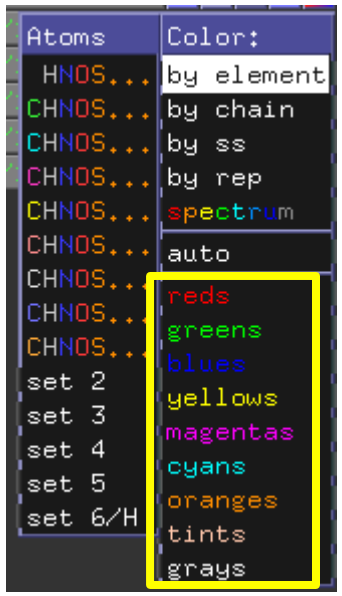
- lines
- spheres
- mesh
- ribbon
- cartoon
- sticks
- dots
- surface

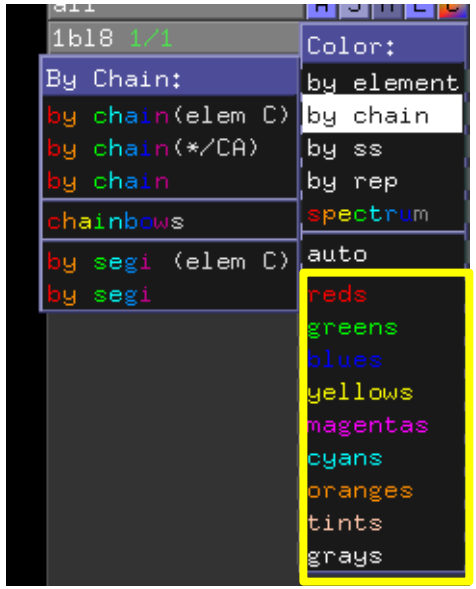
PyMol VIII – Representations II

- When opening the Show Menu of a structure you can see a list of all available representations (yellow)
- By clicking on one of them the corresponding representation of the structure is added to your session
- However you can also use show as (red) to hide all representations and show only the selected one in the smaller menu (blue)



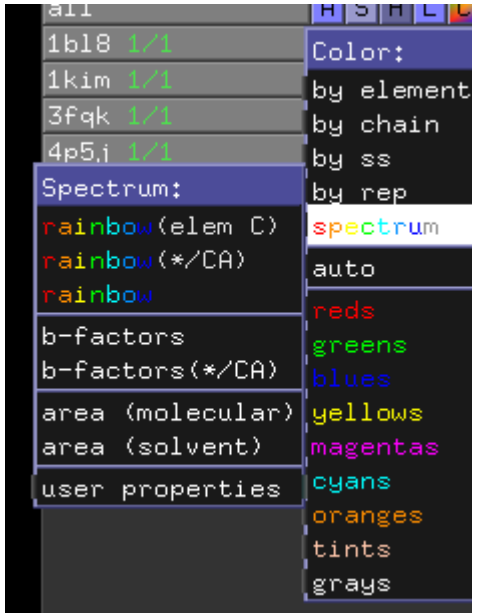
PyMol IX – Colors I

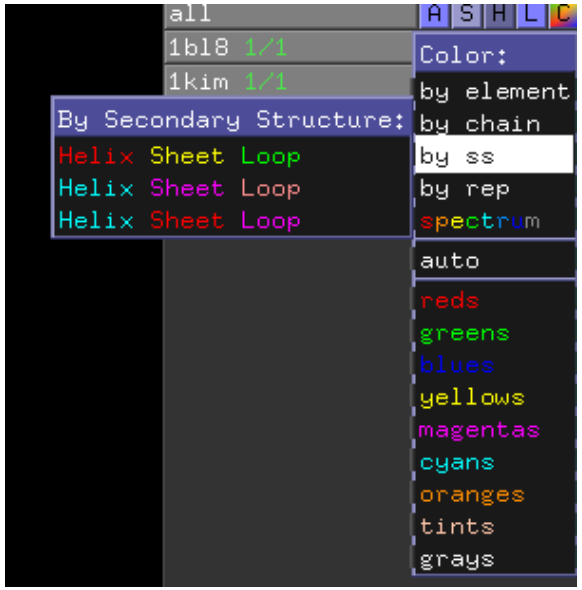




- The color menu allows you to color structures as you see fit
- Aside of the single colors like reds or green (yellow) you can also choose to color according to different categories
- These options include to color according to the element, that means Carbon, Nitrogen, Oxygen and Sulfur atoms all are colored differently (top menu)
- A second option is to color according to chain, that means each macromolecule of your structure is assigned a different color (bottom menu)

PyMol X – Colors II

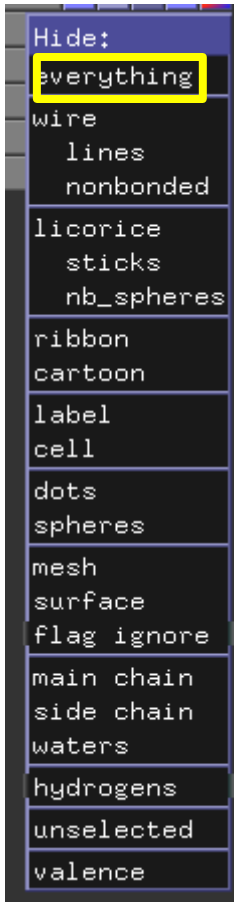




- You can also color according to the secondary structure elements Helix, Sheet, Loop (right example)
- You can color according to a spectrum with blue at the N-terminus to red at the C-terminus (left example)

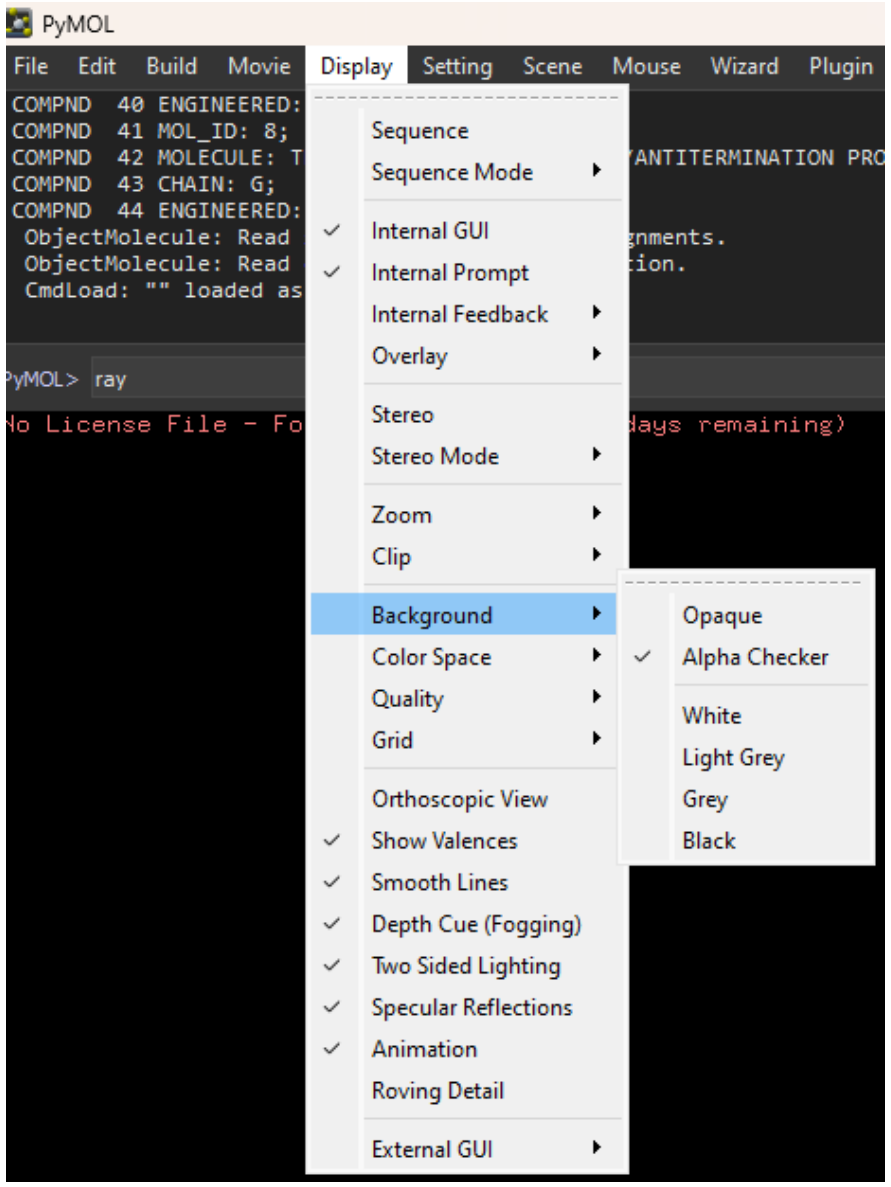
PyMol XI – Hide

- Last but not least you can hide certain representations of our structures with the “Hide” menu
- Aside of hiding single representations we are also able to hide all representations by choosing Hide: everything (yellow)

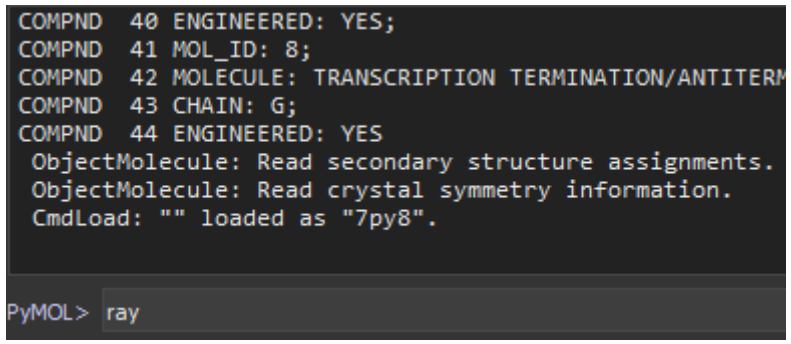


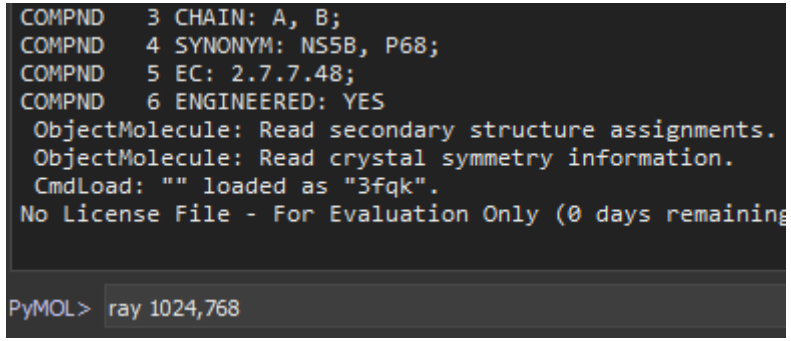
PyMol XII – Image creation I

- Once you‘re finished with coloring your structure and choosing the correct representation you can go with saving images
- The first thing you should do is to switch to a white background with the help of the display menu at the top (see image on the right)
- The options Opaque and Alpha Checker stand for a visible or see-through background respectively



PyMol XIII – Image creation II

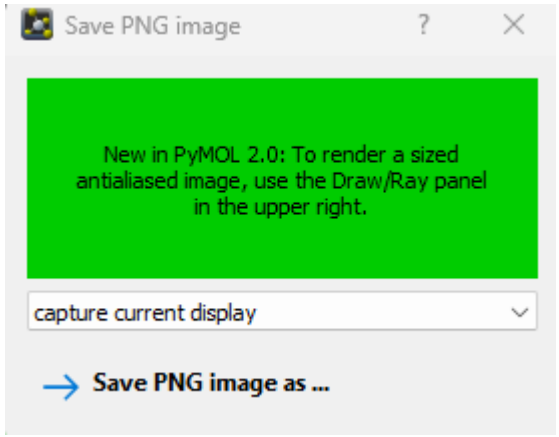


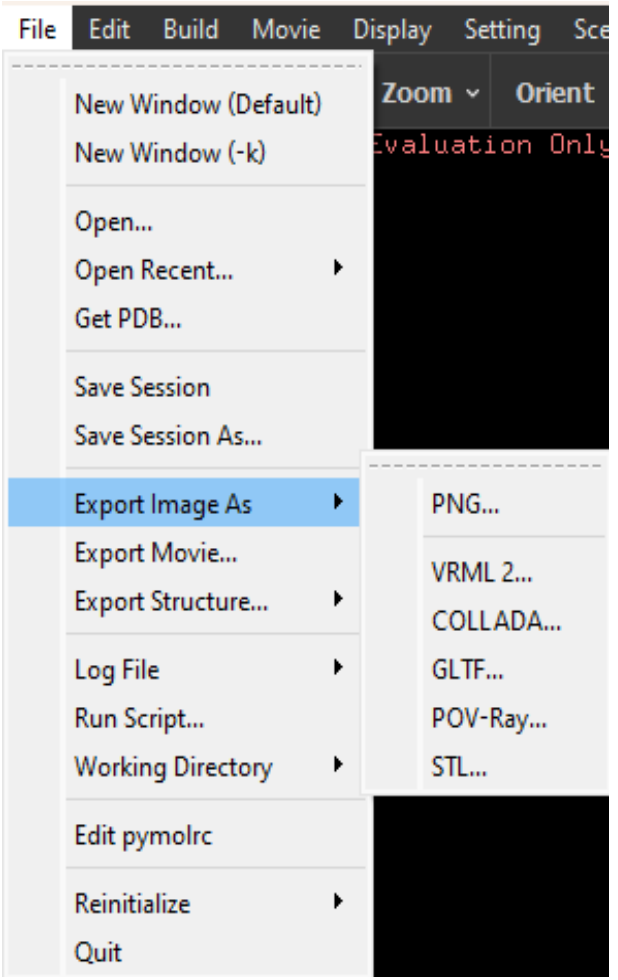


- After you choose the right background you should render the current image
- This can be done with the help of the command ray, which you can type in the command line at the bottom of the session (top image)
- ray allows you to specify the size of the image in pixel as well by adding width and height comma separated afterward (bottom image)

PyMol XIV – Image creation III

- Now you can export whatever you can currently see in you PyMol session to a png-file
- To do so go to the option “Export Image As” in the File Menu at the top and choose “PNG” (right image)
- A Pop-Up window appears (below) and by clicking on “Save PNG image as” you can save the image





Exercise 01: Using PyMol

- Use PyMol to look at the ten different pdb files in Moodle and fulfill the tasks below by saving images of the corresponding structure:
  1. For 3FQK and 7T10 color by chain and create a cartoon representation
  2. For 4P5J and 7Y5I color by elements and create a stick representation
  3. For 7PY8 and 7XCM color by secondary structures and create a ribbon representation
  4. For 6ANF and 7QOV color by spectrum and create a dots representation
  5. For 1KIM and 7UN2 color by chain and atom and create a cartoon and a stick representation

PyMol – Getting Structure Files

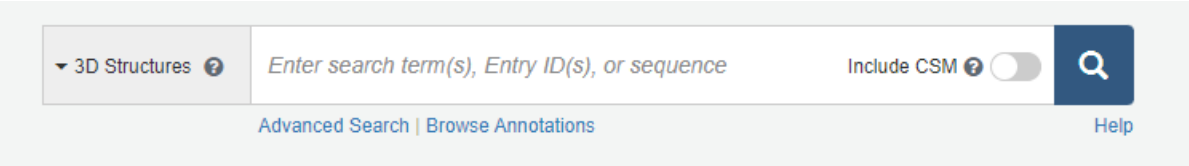
PyMol XV – Getting Structure Files I

- You can get coordinates for protein structures by downloading pdb Files from the RCSB-PDB or other resources like AlphaFold or Modelling server
- Another way is to fetch the coordinates of structures from the RCSB-PDB directly in PyMol with either fetch:

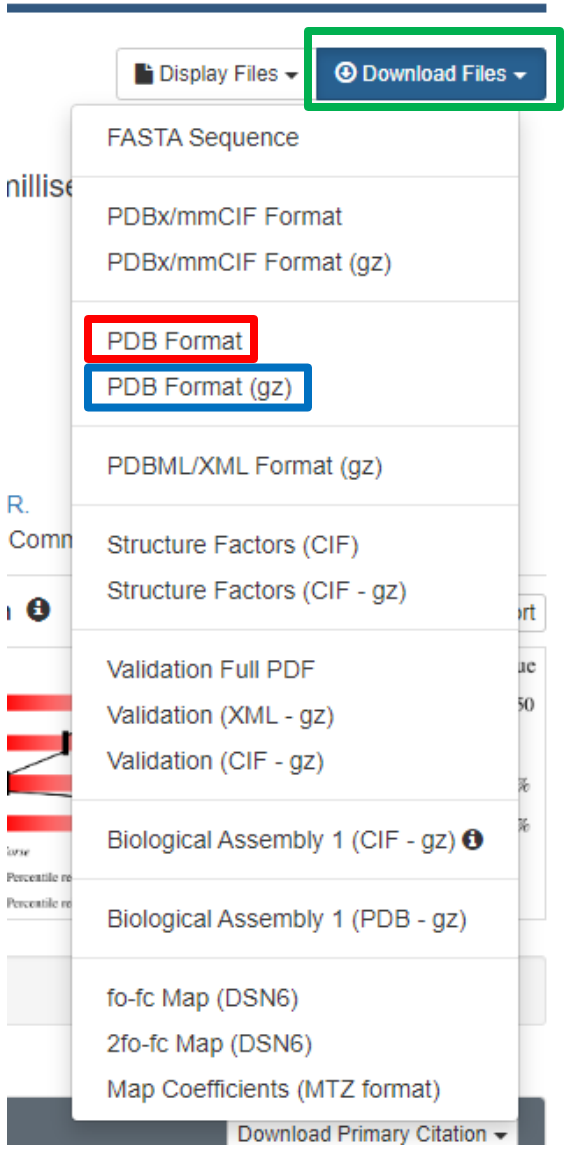
```
fetch 1bl8
```

- Or by using File → Get PDB… as you saw earlier

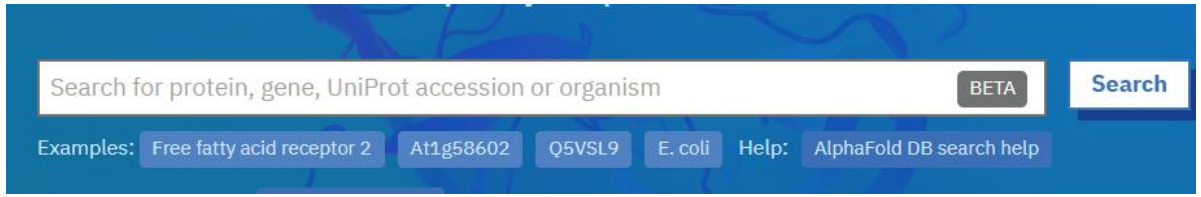
PyMol XVI – Getting Structure Files II

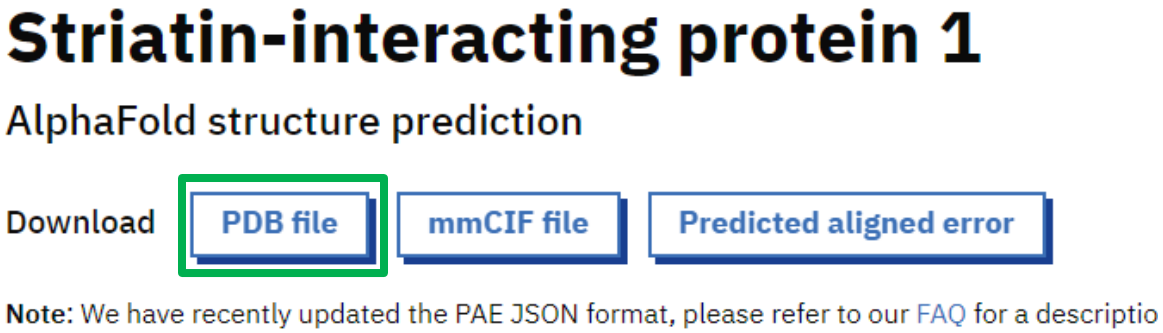


- On the webpage of the RCSB-PDB (https://www.rcsb.org) you‘re able to search for structures according to name(s), identifiers(s) and so on (top)
- Once you opened a site of a singular entry in the RCSB-PDB you can download the structure file under “Download Files” (green) by clicking on either PDB Format (red) to get just the file or PDB Format (gz) to get an archive of the file (blue)



PyMol XVII – Getting Structure Files III





- On the webpage of the AlphaFold database (https://alphafold.ebi.ac.uk) you can search for structures (top image)
- Once you open a site of one specific entry in the database you can download the structure file by clicking on PDB file (botton image, green)

Exercise 02: New pdb files

- Download the structures in the left column from the RCBS-PDB
- Load all of them into a PyMol session and color by chains
- Use the fetch command to load the structures in the right column of the table into a second PyMol session
- Show a Stick representation and color by Element
- Use the UniProt identifiers in the second table to search for structures in the Alphafold database (first result)
- Open all of them in a third session
- Show a Lines representation and color by spectrum
- Create an image at the end for each session

5HJK	1OIU
2ASD	3BVC
3QWE	5SDG
7YXC	7KOL
1ZUI	1BL8
4BNM	4LCZ
6GHJ	2GAS

B4DTL8
A0A5N3XQ52
O14905
P05067
P09850
Q96FZ2

PyMol – Selections

PyMol XVIII – Selections I

- PyMol allows you to create so called selections that contain only atoms in your structure that fulfill certain criteria
- These selections can be used to look at only parts of your structure, for example only certain amino acids, or just the N-terminus
- You can create selections based on many different things:
  - Chains
  - Specific Amino Acids
  - Specific Atoms
  - Ligands

PyMol XIX – Selections II

- When working with Selections we need to remember how our pdb-files look like, especially how the coordinate section looks like
- As we can see this section contains tabular data and our selections will pick the coordinates of all atoms that fulfill the corresponsing criteria

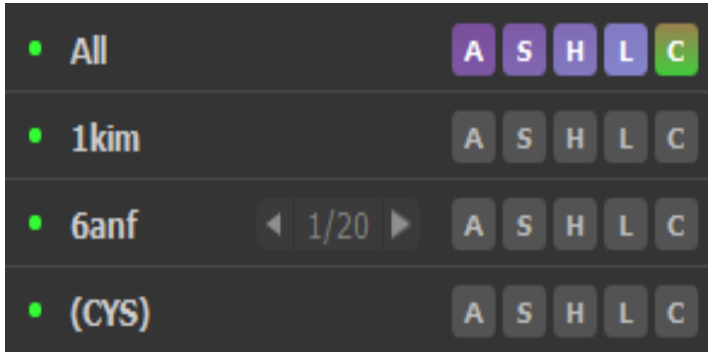
```
ATOM    841  N   CYS A 171      38.741  77.834  55.215  1.00 11.75           N
ATOM    842  CA  CYS A 171      39.671  77.832  56.336  1.00 12.29           C
ATOM    843  C   CYS A 171      40.108  79.166  56.902  1.00 12.21           C
ATOM    844  O   CYS A 171      39.952  79.408  58.087  1.00 10.41           O
ATOM    845  CB  CYS A 171      40.858  76.933  56.017  1.00 11.97           C
ATOM    846  SG  CYS A 171      40.345  75.276  55.592  1.00 12.82           S
```

PyMol XX – Selections III

- We can create Selections by writing our select statements into the command line at the bottom of our session (below)
- If a selection was successful you get a selector message (top right) and a Selection object appears in the sidebar (bottom right)

```
PyMOL>select CYS, resn cys
 Selector: selection "CYS" defined with 192 atoms.
```

```
COMPND   34 MOL_ID: 7;
COMPND   35 MOLECULE: DNA-DIRECTED RNA POLYMERASE SUBUNIT OMEGA;
COMPND   36 CHAIN: E;
COMPND   37 SYNONYM: RNAP OMEGA SUBUNIT,RNA POLYMERASE OMEGA SUBUNIT,
COMPND   38 TRANSCRIPTASE SUBUNIT OMEGA;
COMPND   39 EC: 2.7.7.6;
COMPND   40 ENGINEERED: YES;
COMPND   41 MOL_ID: 8;
COMPND   42 MOLECULE: TRANSCRIPTION TERMINATION/ANTITERMINATION PROTEIN NUSG;
COMPND   43 CHAIN: G;
PyMOL> select CYS,resn cys
```



PyMol XXI – Selections IV


```
select ChainA, chain A
select HIS, resn HIS
select CA, name CA
select nterm, resi 1-10
select Helix, ss H
select HelixA, chain A & ss H
select ntermAt, id 1-50
```

- On the left side you can see some example selections
- The general syntax is always as follows:

select selection_name, criterium value

- So the first example creates a Selection object called ChainA by searching for atoms that have the chain identifier A, that means “chain” is the criterium and “A” the value

PyMol XXII – Selections V [13]

- We can use any of the seven criteria shown in the table below
- Furthermore we can combine multiple criteria using and/or

Statement	Description
symbol	Means the atom symbol in the last column of pdb files, e.g. O or N
name	Refers to the atom name, e.g. CA or CB
resn	Refers to the name of the residue, e.g. HIS or ALA
resi	Refers to the residue id or residue number, e.g. 200 or 250
chain	Refers to the protein chain, e.g. A or B
ss	Refers to the secondary structure of the protein H: Helix S: Sheet L: Loop “”: Unstructured
id	Refers to the Atom number, is not very practical as long as you don‘t know the exact number of atoms in your selection

PyMol XXIII – Selections VI

- When creating Selections we are able to pick multiple values at once
- When working with the criteria name or resn we can do so by using + between different atom and/or residue names:

```
select proteinbackbone, name CA+C+N+O
```

```
select Aromatic, resn HIS+PHE+TYR+TRP
```

- When working with the criteria resi or id we can use ranges:

```
select nterm, resi 1-10
```

```
select ntermAt, id 1-50
```

```
select Test, id 1-50+100-150
```

PyMol XXIV – Selections VII

- Per default Selections in PyMol are executed on all structures / objects available in your current session
- If you want to select only parts from a specific structure you need to use the name of this structure as one of the criteria
- Let‘s say you want to select only Tryptophanes in the structure 3FQK, than you would need the following selection:

```
select TRP_3FQK, 3FQK & resn TRP
```

- Instead of a structure you can also use a selection object:

```
select CA_TRP_3FQK, TRP_3FQK & name CA
```

Exercise 03: Selections in PyMOL

- Create the selections shown in the table below in PyMol (pick one of the structures).

Selection	**Structures**
All CG Atoms, including CG1 and CG2	1KIM, 7TF2, 7XCM, 1BL8
All atoms in β-sheets	7PY8, 7QR2, 7T10, 3FQK
All atoms in helices	7T10, 1BL8, 3FQK, 4P5J
All atoms in unstructured regions	7PY8, 3FQK, 7TF2, 7QOV
All atoms of aromatic amino acids (TYR, PHE, HIS, TRP)	7QOV, 3FQK, 1BL8, 1T10
All atoms of Glutamines (GLN) and Glutamates (GLU)	1BL8, 1KIM, 1QOV, 7TF2

PyMol – Scripts

PyMol XXV – Scripts I [14]

- PyMol allows you to run scripts containing multiple PyMol commands, similar to Bash scripts to make sure everything is executed in the same manner over and over again
- This can be especially useful when working with multiple different structures of one and the same protein, to be able to create the exact same image over and over again
- Furthermore it can help you to recreate the same images again and again, without you typing in all commands by hand or making all changes by hand using the menus
- PyMol scripts should get the extension .pml

PyMol XXVI – Scripts II [14]

- In PyMol we can do everything we learned so far with the help of PyMol commands instead of using these menus
- However in most cases, especially when working only sporadically with PyMol using the menus is easier than using commands
- Nevertheless we will discuss at least some commands you can use in scripts on the next slides

PyMol XXVII – Scripts III

- The first command can be used to reset your current session to an empty one
- This can be achieved by using the following command:

```
reinitialize
```

- When writing PyMol Script you should always start with this command to make sure you start with an empty session and with all settings back to default values

PyMol XXVIII – Scripts IV – Show I

- The command show can be used activate the different representations of our structures
- The general syntax is as follows:
  - show representation
- So in the first example we activate the cartoon representation for all structures
- If you want to activate a representation for just one structure or selection you can add the name after the command
  - show representation, Objectname

```
show cartoon
show lines, Model1
show_as sticks
hide everything
show dots, ss h
```

PyMol XXIX – Scripts V – Show II

- So in the second example we activate the lines representation only for the structure Model1

```
show cartoon
```

- You can also change between representations by using show_as:
  - show_as representation, Objectname

```
show lines, Model1
```

```
show_as sticks
```

- So in the third example we switch to a stick representation for all structures

```
hide everything
```

```
show dots, ss h
```

PyMol XXX – Scripts VI – Show III

- We can also hide certain representations:
  - hide representation, Objectname
- Or we can hide all representations using:
  - hide everything, Objectname
- Last but not least we can use statements similar to our conditions for selections to activate representations only for parts of our structures
  - show representation, criterium value
- So in the last example we show a dots representation only for Helices

```
show cartoon
```

```
show lines, Model1
```

```
show_as sticks
```

```
hide everything
```

```
show dots, ss h
```

PyMol XXXI – Scripts VII – Color I

- Another useful command is color, that allows us to switch between different coloring options
- The general syntax is:
  - color color_name
  - color atomic
- So in the first example I color all structures red and in the second one I color them according to their atoms

```
color red
```

```
color atomic
```

```
color yellow, ss h
```

```
color red, (1kim and name C*)
```

```
color cyan, (1kim and chain A)
```

```
util.cbay
```

PyMol XXXII – Scripts VIII – Color II

- Again we can use statements similar to color only parts of our structures:
  - color colorname, criterium value
- So in the third example I color all Helices yellow

```
color red
```

```
color atomic
```

```
color yellow, ss h
```

```
color red, (1kim and name C*)
```

```
color cyan, (1kim and chain A)
```

```
util.cbay
```

PyMol XXXIII – Scripts IX – Color III

- If you want to use multiple criteria you need to put them in brackets () and combine them using and/or
- So in the fourth example I color all carbon atoms (name C*) of the structure 1kim red
- Or I color the first macromolecule (chain A) of the structure 1kim in cyan

```
color red
```

```
color atomic
```

```
color yellow, ss h
```

```
color red, (1kim and name C*)
```

```
color cyan, (1kim and chain A)
```

```
util.cbay
```

PyMol XXXIV – Scripts X – Color IV

- A last Advanced coloring method is to use the util.cba* (color by atom) command
- The asterisk can be replaced by a short letter for a color of your choice (here y for yellow)
- This color is asigned to carbon atoms, while oxygen atoms are colored red, nitrogen atoms blue and so on

```
color red
```

```
color atomic
```

```
color yellow, ss h
```

```
color red, (1kim and name C*)
```

```
color cyan, (1kim and chain A)
```

```
util.cbay
```

PyMol XXXV – Scripts XI – Background

- After you set up all representations, created all needed selections and colored everything as you want, you can now switch the background to a color of your choice:
  - bg_color colorname
- For example to get a white background you could use:

```
bg_color white
```

PyMol XXXVI – Scripts XII – Render and Image

- Once all settings are to your liking you need to render the image using the command ray:

```
ray
```

- Afterwards you can save a png image with the command png:
  - png Imagename.png
- So for example you could create an Image called Script01.png as follows;

```
png Script01.png
```

PyMol XXXVII – Scripts XIII – Center and Zoom

- If you want a certain Object to be in the middle of your image you can use the command center:
  - center Objectname
- Let‘s say you have an Object called Ligenv and you want to have this in the middle of your image:

```
center Ligenv
```

- If you want to look at something more in detail you can use zoom to enlarge a certain object:
  - zoom Objectname
- Again with the Object called Ligenv we can enlarge our view on it as follows:

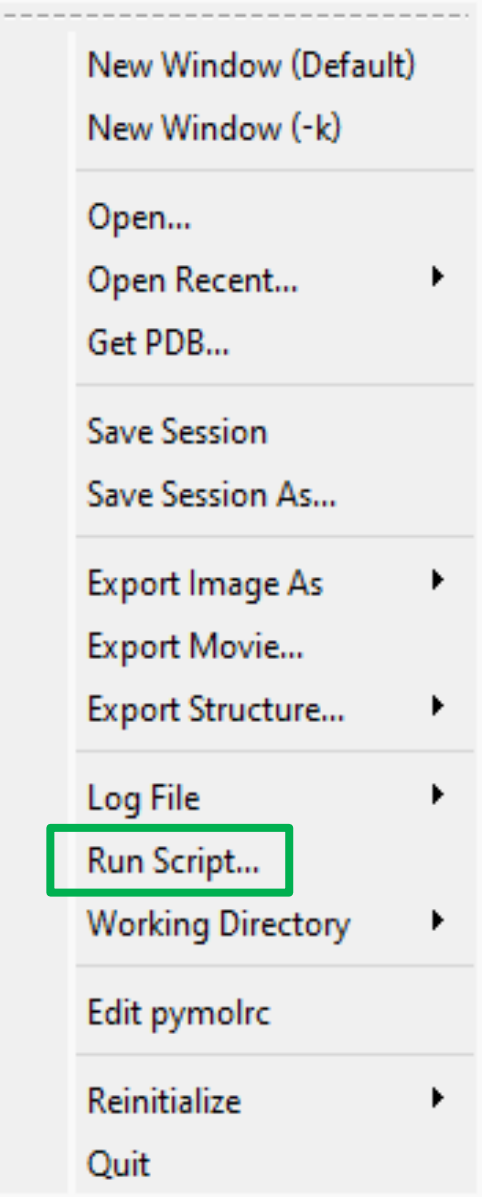
```
zoom Ligenv
```

PyMol XXXVIII – Scripts XIV

- For more information about available commands you can use the PyMol wiki as a starting point
- In there you will find a category called commands under the following link:
  - https://pymolwiki.org/index.php/Category:Commands
- In there you find an alphabetical overview of many (if not all) commands available to you

PyMol XXXIX – Scripts XV

- After you created your script you can execute it in PyMol
- To do so you need to go to the file menu at the top of your session and in there click on Run Script (green rectangle)
- A File Dialog opens and you can pick the file you want to execute



Exercise 04: PyMol Scripts

- Write a PyMol Script that downloads the structure 7Y5I from the RCSB-PDB and fulfills the following tasks with it:
  - Create a cartoon representation and color it red
  - Create an image of the centered structure
  - Hide the structure
  - Create a selection of the Ligand TLA and create a stick representation of it
  - Create an image of the centered and zoomed in Ligand
  - Do the same for the Ligand ARG in the chain A with a residue id of 402 (resi 402)

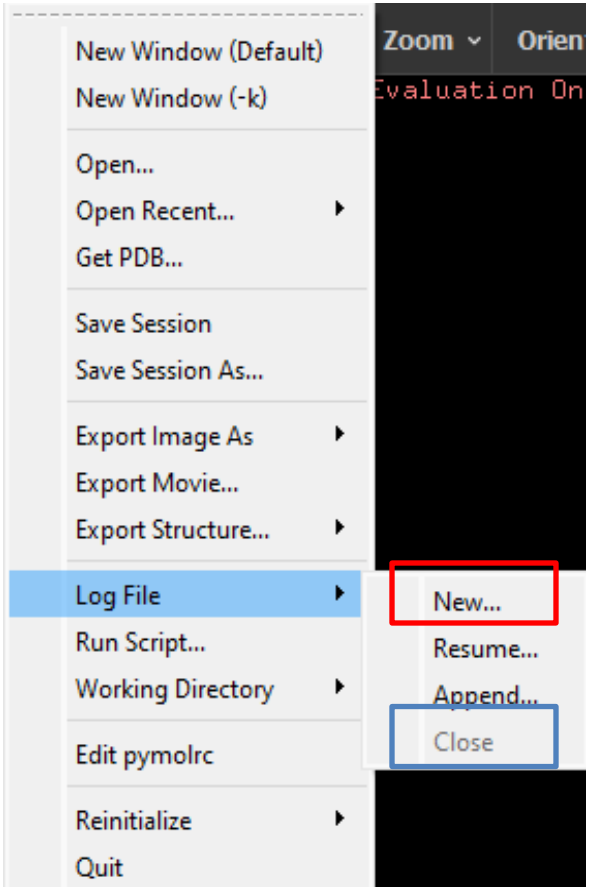
PyMol – Log Files

PyMOL XL – Log Files I

- A simple way of creating PyMol scripts and making your sessions repeatable are log-files
- These files record everything you do in the session for as long as they are open
- Once you‘re done with your session you can close the log-file and everything is saved
- Log files can be especially useful to keep track of what you did
- A normal session will save only the end result, but the log file can show you how you got there

PyMOL XLI – Log Files II

- There are two ways to open / close log-files
- The first way is to use the file menu (image) and go to Log File and in the submenu there click on New… (red rectangle)
- Now a file dialog opens were you specify where to save and how to name the log-file
- Once your done go back to this menu and click on Close (blue rectangle) to save all changes



PyMOL XLII – Log Files III

- The second option is to use the command log_open
- The syntax of this command is:
  - log_open filename
- That means with the command below you can create a log-file called „Session_2024_01_26.pml“:

```
log_open Session_2024_01_26.pml
```

- Once you‘re done you can use the command log_close to close the log-file and save all changes

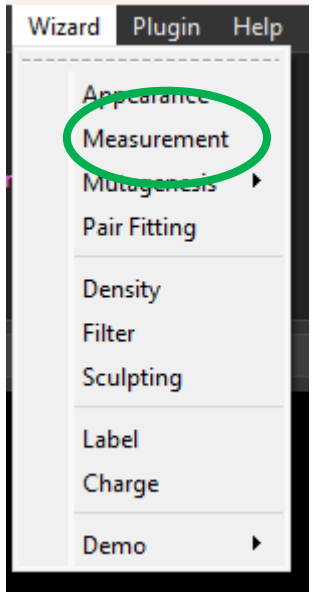
Exercise 05: A Simple Log-File

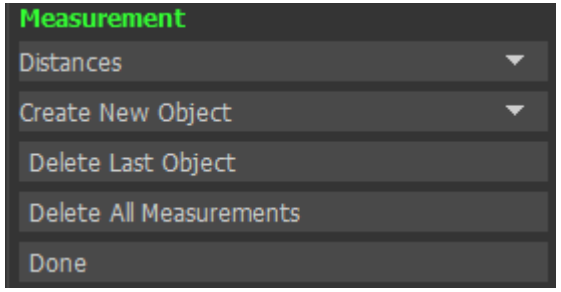
- Open a new PyMol Session
- Open a log-file and then execute the following tasks:
  - Download the PDB-Structure 8I35 via fetch
  - Hide everything and show only a cartoon representation of the structure
  - Color the structure by chain and save a first image
  - Create a selection of the chain A, and show only this selection in a stick representation
  - Save a second image
  - Show the whole structure as a surface representation and save a third image
- Close the log-file at the end

PyMol – Measurements

PyMol XLIII – Measurements I

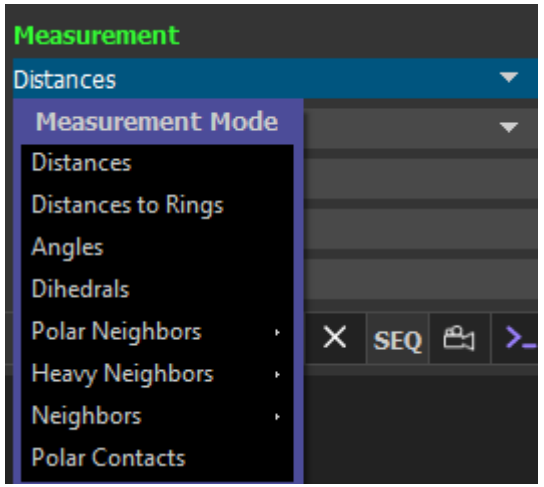
- Aside of Visualizing structures PyMol allows us to measure things like distances, angles and dihedrals
- You can find the measurement tool in the top menu under Wizard and there you have to choose the option measurement (right image, green)
- A measurement wizard appears in your sidebar (bottom image) and by clicking on “Distances” you can switch between different modi





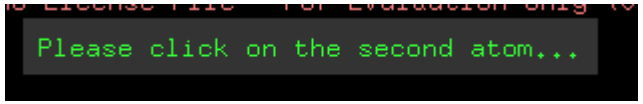
PyMol XLIV – Measurements II

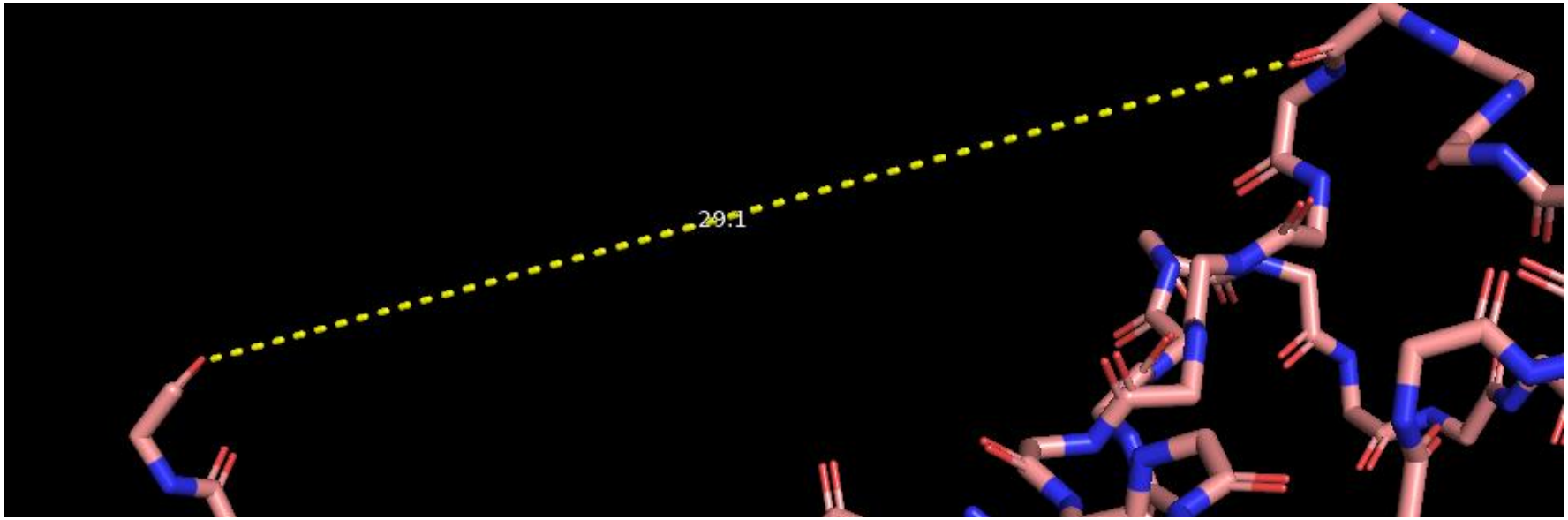
- The Wizard has eight different modi available:
  - “Distances” to measure the distance between two atoms
  - “Distances to Rings” to measure the distance between centers of aromatic rings and atoms
  - “Angles” to measure the angle three atoms form in the 3D-realm
  - “Dihedral” to measure the rotation angle around a bond defined by four atoms
  - Some more obscure measurements for neighboring atoms



PyMol XLV – Measurements III – Distances

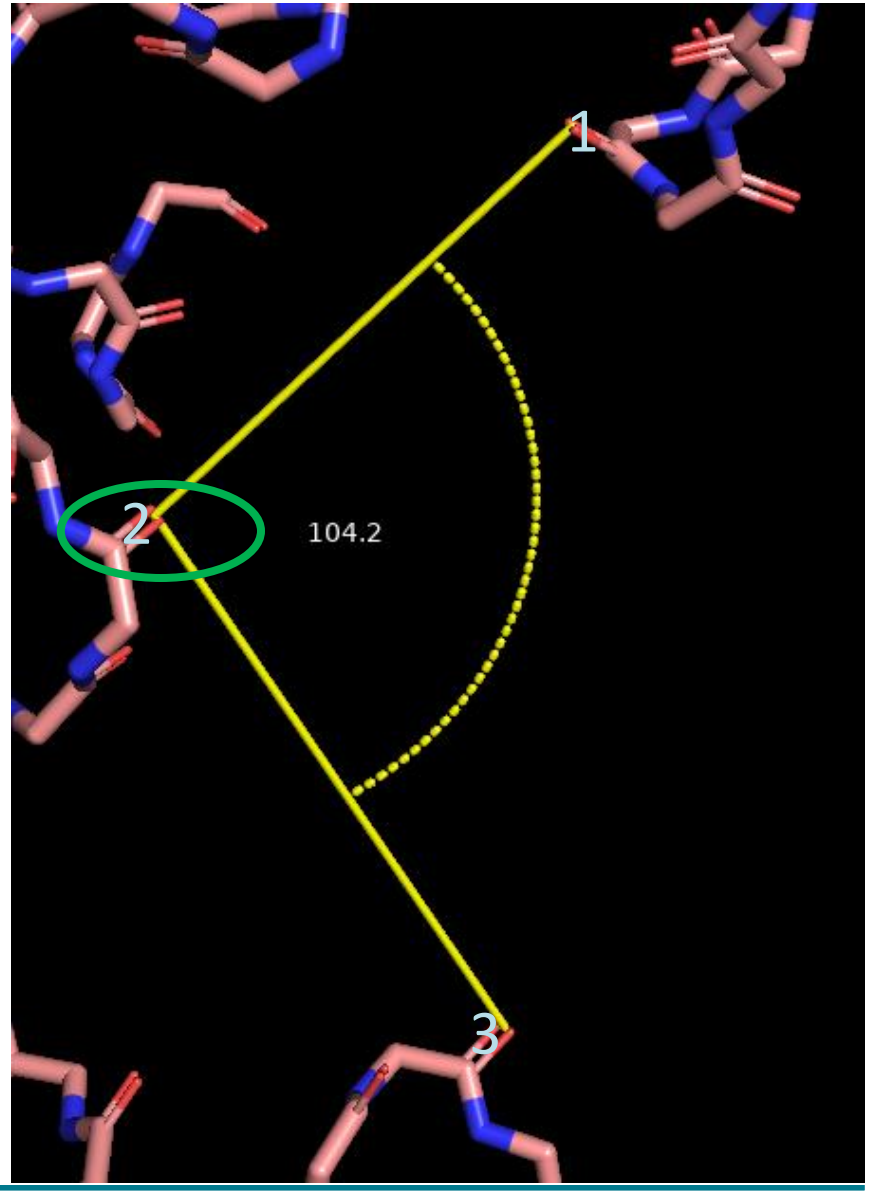
- Once you activated the Wizard you‘re able to start the measurements by clicking on two atoms
- The wizard will show a message at the top right corner of your session to tell you what you need to do (right)
- After you clicked the second atom a measurement line including the distance in Å, here 29.1Å will appear (bottom)





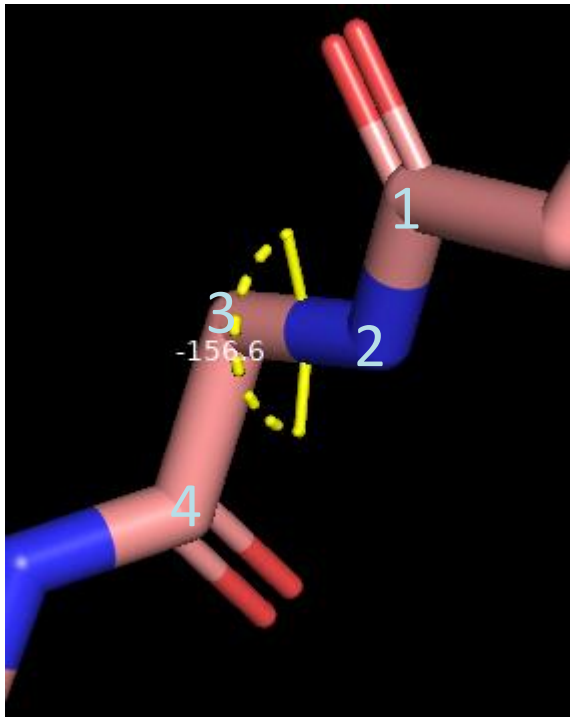
PyMol XLVI – Measurements IV – Angles

- To measure angles we need to click on three atoms consecutively (numbers)
- You need to keep in mind that the second you click on is the central point of your angle (green)
- Afterwards the two sides of your angle appear as yellow lines and a dotted bow is drawn indicating the measured angle
- The angle value itself appears in degree, here 104.2°, between the bow and the two sides



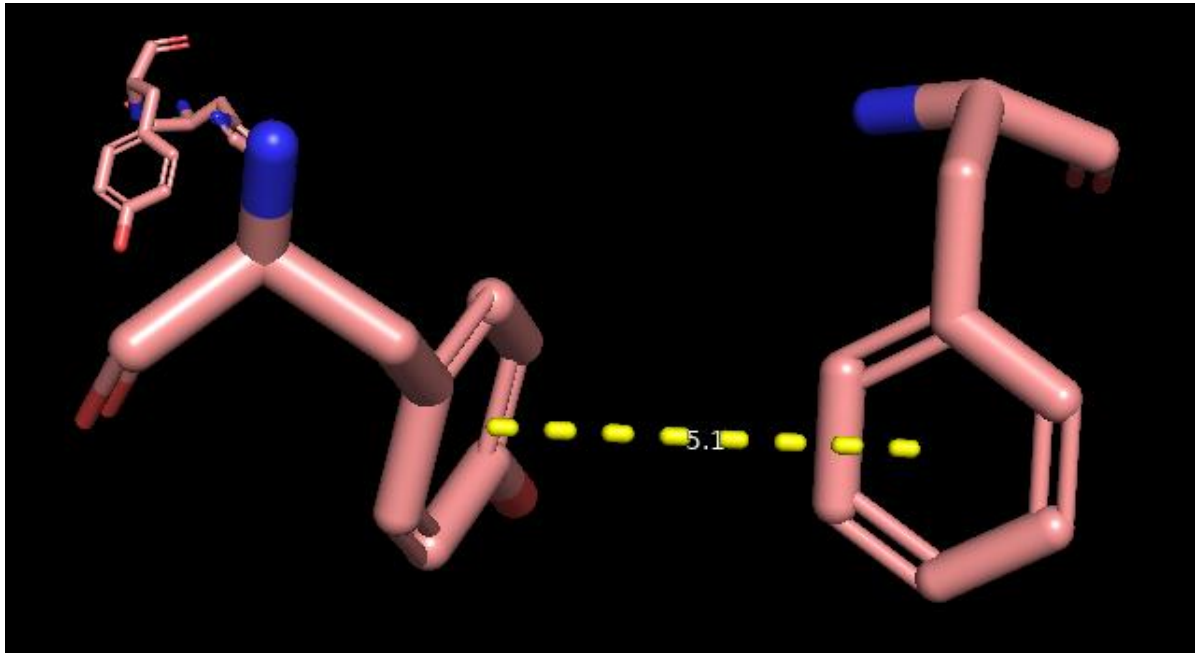
PyMol XLVII – Measurements V – Dihedrals

- Dihedrals or rotation angles over chemical bonds play an important role in protein structures
- To measure them we need to click on four atoms (numbers), with atoms number 2 and 3 forming the bond we are interested in
- Afterwards two small yellow lines will appear on either side of the bond and a dotted bow will be drawn between them
- The angle will than again appear in degree, here -156.6°



PyMol XLVIII – Measurements VI – Ring distances

- The distance of atoms to aromatic rings or between aromatic rings can be measured as well
- Here we simply need to click on one atom of our ring(s) and the distance to the central point of this ring(system) is measured
- Again the measurement appears as dotted yellow line with the distance in Å, here 5.1Å will appear



Exercise 06: Measuring in PyMol

- Use the Measurement Tool in PyMol to measure the following in the structure 7T10:
  - Distances between SG atoms of Cysteines
  - Distances between the nitrogen atoms of Histidine (ND1 and NE2)
  - The CA-CB-SG angle in Cysteines
  - The NE-CZ-NH1 and NE-CZ-NH2 angles in Arginines
  - The CB-CG-CD-NE dihedral in Arginines
  - The CG-CD-CE-NZ dihedral in Lysines
  - Distances between the ring systems of aromatic amino acids (Histidine, Phenylalanine, Tyrosine and Tryptophan)

PyMol XLIX – Measurements VII – Commands I

- Aside of using the measurement wizard you can also use commands for measuring distances, angles and dihedrals
- The commands are:
  - distance ,selection1, selection2
  - angle ,selection1, selection2, selection3
  - dihedral ,selection1, selection2, selection3, selection4
- The corresponding selections can either be single atoms or multiple atoms

PyMol L – Measurements VIII – Commands II

- If you want to measure the distance between two CA-atoms you could use the follwing command:

```
distance ,resi 10 and name CA and chain A, resi 40 and name CA and chain A
```

- If you want to measure the distance between 1 CA-atom and 8 other CA-atoms you could use:

```
distance ,resi 10 and name CA and chain A, resi 35-42 and name CA and chain A
```

PyMol LI – Measurements IX – Commands III

- To measure the angle of the three backbone-atoms of a certain amino acid you could use:

```
angle ,resi 60 and name CA and chain A, resi 60 and name C and chain A, resi 60 and name N and chain A
```

- To measure a dihedral angle in the sidechain of an Arginine you could use:

```
dihedral ,resi 32 and name CB and chain A, resi 32 and name CG and chain A, resi 32 and name CD and chain A, resi 32 and name NE and chain A
```

PyMol LII – Measurements X – Commands IV

- Since these commands can become rather large very quickly, especially when you pick only single atoms, it might be better to create some selections first and use them for the measurement afterwards:

```
select atom1, resi 544 and name CB and chain A
select atom2, resi 544 and name CG and chain A
select atom3, resi 544 and name CD and chain A
select atom4, resi 544 and name NE and chain A

dihedral ,atom1, atom2, atom3, atom4
```

PyMol LIII – Measurements XI – Commands V

- You can always name your measurements as you see fit by adding a name after the command and before the first comma:

```
dihedral ARG_544 ,atom1, atom2, atom3, atom4
```

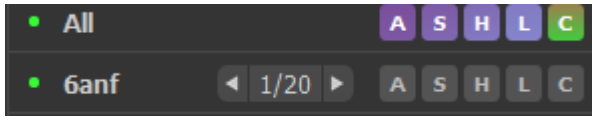
Exercise 07: Measurement script

- Write a PyMol Script that downloads the structure 7QR2 and measures the the thing shown in the table below
- The atoms are written with their atom name and the residue number in brackets

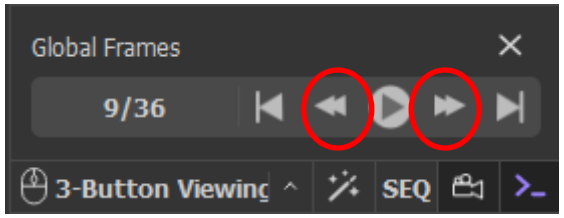
Distances	Angles	Dihedrals
CA (72) – CA (87)	S (501) – S (502) – S (503)	O1 (505) – C1 (505) – C2 (505) – O2 (505)
CA (353) – CA (370)	S (501) – S (502) – S (504)	O1 (506) – C1 (506) – C2 (506) – O2 (506)
S (501) – S (502)	S (501) – S (503) – S (504)	O1 (507) – C1 (507) – C2 (507) – O2 (507)
S (501) – S (503)	S (502) – S (503) – S (504)	
S (501) – S (504)		
S (502) – S (503)		
S (502) – S (504)		
S (503) – S (504)		

PyMol – States

PyMol LIV – States I



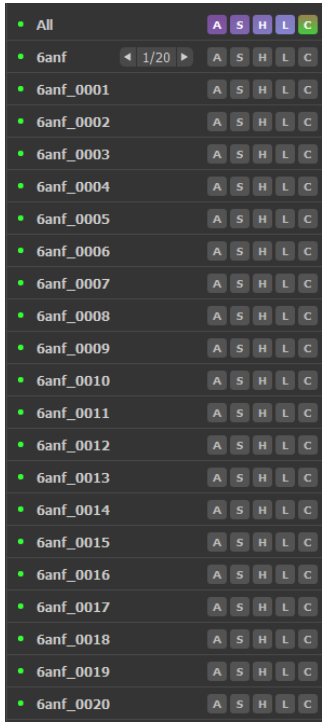
- When working with the structure 6ANF earlier you could see a 1/20 beside the name of the structure (top image)
- This refers to different states (models) of the structure present in the pdb-file
- This is something, that is especially prevalent when dealing with NMR-structures and allows us to compare different conformations

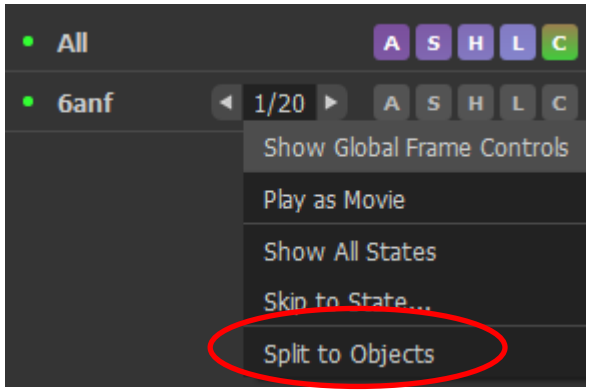


- You can switch between different states with the „global frames-menu“by clicking on the different arrow-buttons (bottom image, red circles)

PyMol LV – States II

- You can also split up the structure into it‘s different states (right image, red circle) by clicking on the numbers beside the name of the structure
- Afterwards you have multiple copies of the structure, one for each state, available in the sidebar (left image)



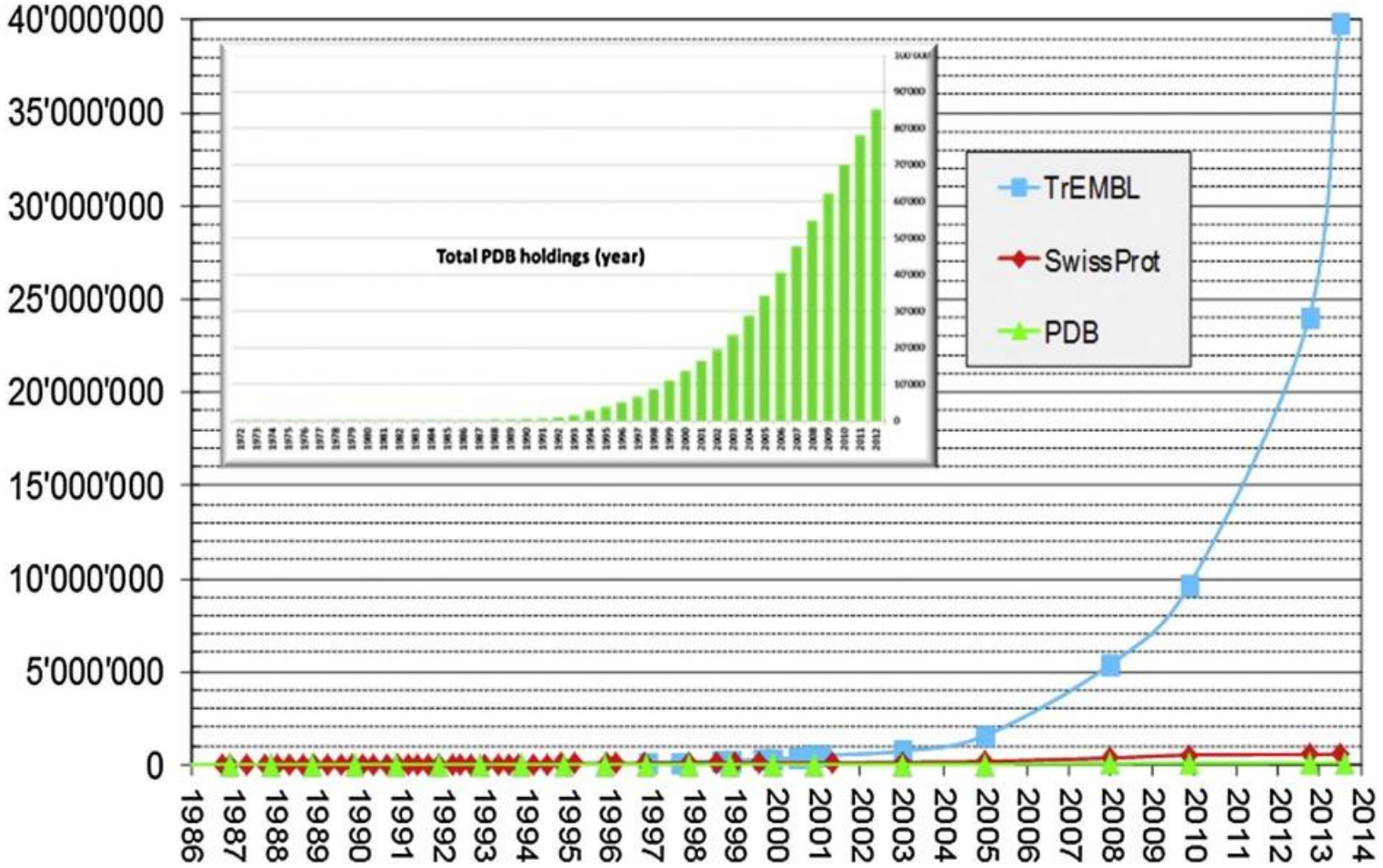


Modelling

Modelling I – Protein-Structure Gap

- The number of known protein sequences increases exponentionally during the last years
  - *Human Genome Project*
- Finding protein structures with the help of experimental methods like X-Ray crystallography, Cryo-EM or NMR techniques can‘t cope with this growth
- Generating protein structure models with the help of computational methods (*in silico*) can be an alternative

Modelling II – Growth of databases [16]



Modelling III – Structure Sequence Relationship [17]

- In his Nobel Price Lecture Christian Anfinsen postulated that the structure of a protein should be dependent only on it‘s amino acid sequence
- While studying the renaturation of ribonuclease he found out, that to get to an active conformation only one of 105 possible combinations of the pairings of the eight sulfhydryl groups would lead to a stable protein structure
- Anfinsen‘s Hypothesis:
  - The three-dimensional structure of a native protein in its normal physiological milieu is the one in which the Gibbs free energy of the whole system is lowest; that is, that the native conformation is determined by the totality of interatomic interactions and hence by the amino acid sequence, in a given environment.

Modelling IV – Levinthal‘s Paradox [18]

- Protein structures have many degrees of freedom to fold to their native structure
- E.g. a protein of 150 amino acids has 450 degrees of freedom, 300 from dihedral angles and 150 because of bond angles in the amino acid side chains
- That leads to the problem that a protein has around 10^{300} possible conformations
- Anfinsen‘s Hypothesis and Levinthal‘s Paradox lead to the formulation of the protein folding problem in structural biochemistry

Modelling V – The protein Folding Problem [19]

- The “protein folding problem” consists of three closely related puzzles:

(1) What is the folding code?

(2) What is the folding mechanism?

(3) Can we predict the native structure of a protein from its amino acid sequence?

- It seems that with the development of AlphaFold at least the third puzzle was solved

Modelling VI – Methods I

- There are three distinct methods of protein structure modelling:

1. Homology modelling
2. Threading
3. Ab initio modelling

- While the first two are similar to each other and make use of knowledge about already determined protein structures the third one differs fundamentally

Modelling VII – Methods II – Homology Modelling

- Homology modelling uses a known protein structure, e.g. from X-Ray crystallography as a template, and is based only upon sequence alignment
- The unknown sequence is aligned to the sequence of the template
- The protein backbone is than generated by using coordinates of the template as a blue-print
  - Since Gaps in the Alignment should lie in unstructured regions, there shouldn‘t be any problems
  - All gaps are filled by using a knowledge-based approach to model these regions
- After the protein backbone is created, the side chains are added to the structure
  - Again the coordinates of the template can be used as a partial blue-print

Modelling VIII – Methods III – Threading

- Is similar to Homology Modelling because it needs a template as well
- While for Homology Modelling the template is chosen based on the sequence alignment, the template for Threading is chosen based on structural data as well as the sequence alignment
- Can use multiple templates together, e.g. for different domains in your sequence
- The remaining process is done similiar to Homology modelling

Modelling IX – Methods IV – Ab initio Modelling

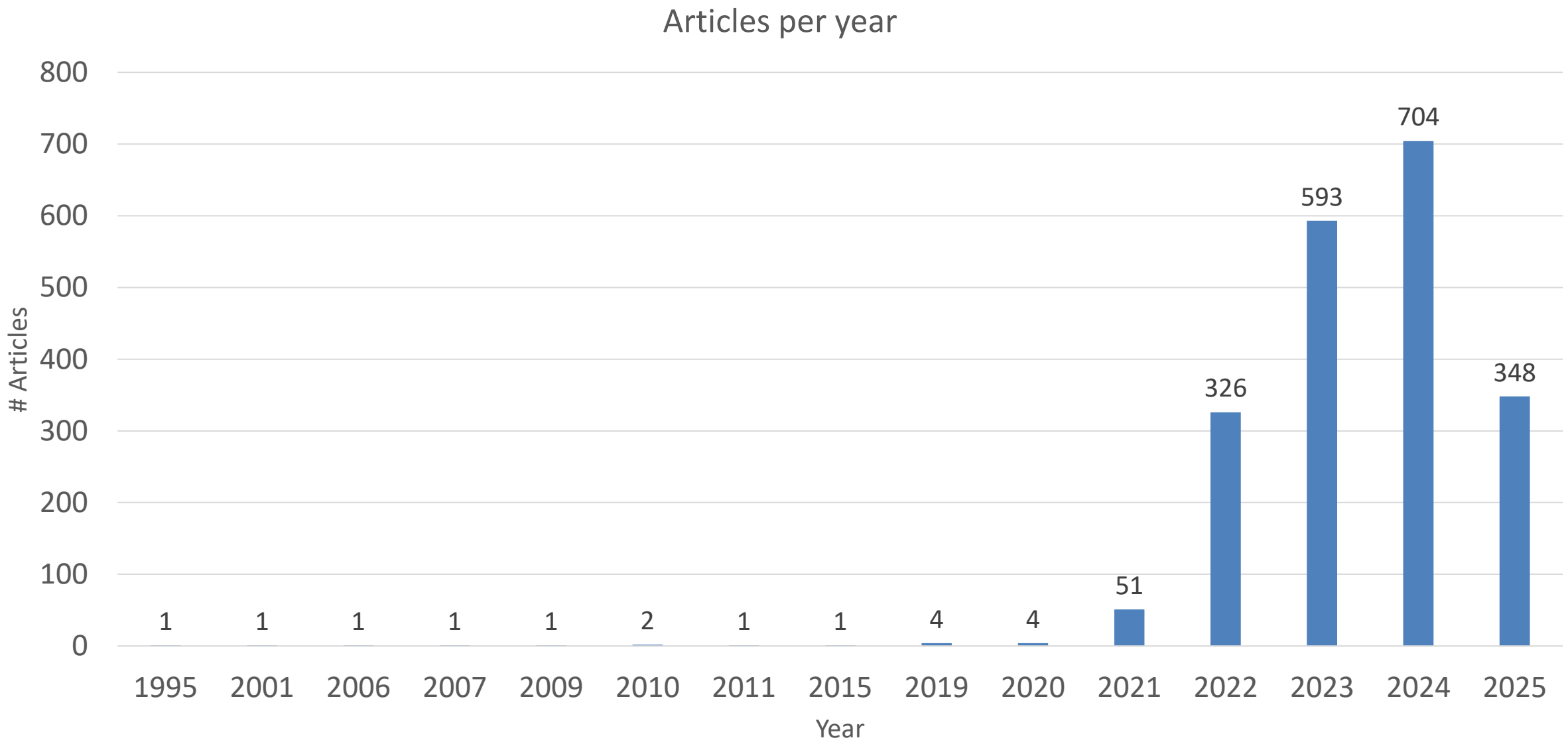
- Creates a protein structure without a template
- The structure is generated from scratch with the help of energy functions
- Very time and resource consuming
- Only feasible for small proteins

Modelling X – AlphaFold I [20]

Modelling XI – AlphaFold II [21]

- The state-of-the-art AI system developed by DeepMind
- Is able to computationally predict protein structures with unprecedented accuracy and speed
- In partnership with EMBL's European Bioinformatics Institute (EMBL-EBI), they released over 200 million protein structure predictions by AlphaFold
- These are freely and openly available to the global scientific community
- Included are nearly all catalogued proteins known to science

Modelling XII – AlphaFold III – PubMed (14.04.2025)



Modelling XIII – AlphaFold IV – CASP I [22]

- Critical Assessment of Structure Prediction
- A competition that is held every two years with groups from all over the world
- Was dominated for years by the Zhang-Lab with I-TASSER and the Baker Lab with Rosetta
- Groups have to create models for proteins where the structure was experimentally solved but not published yet

Modelling XIV – AlphaFold V – CASP II

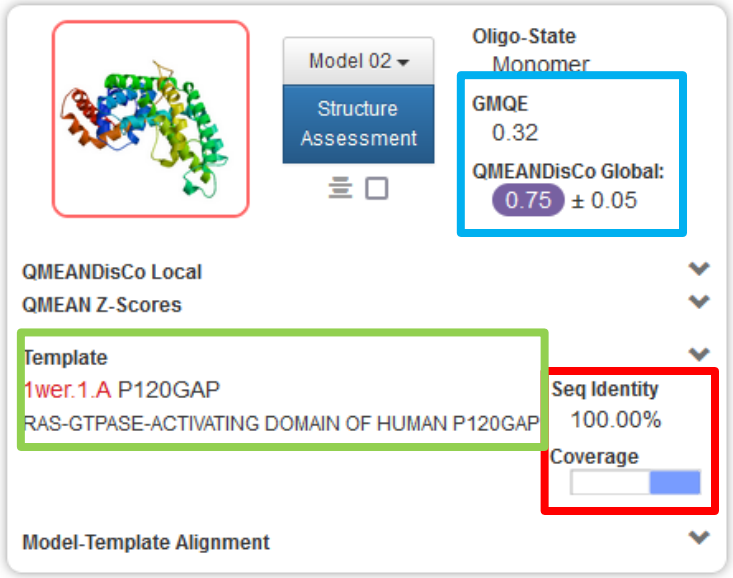
- First Version scored around 70 – 75 (of 100) in 2018 [23]
- The general mean of results for programs and servers was 60 [23]
- In 2020 AlphaFold 2 scored an average of 90, results were available for the public in 2021 [24]
- Since it dominated the competition and it was declared that there is no difference between a structure calculated by AlphaFold and experimental methods, they refrained from joining the competition in 2022

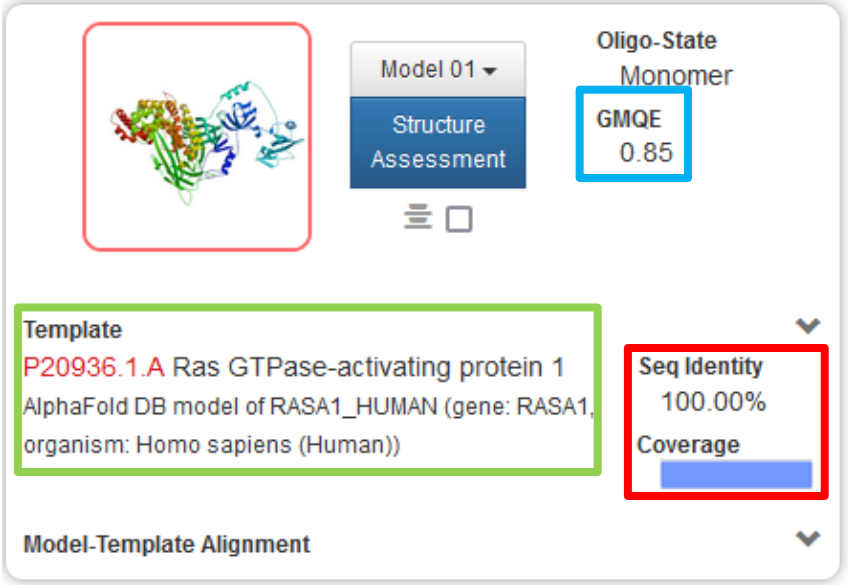
Modelling XV – SwissModel I

- One web page you can use to generate Homology Models is the SwissModel Repository (https://swissmodel.expasy.org/interactive)
- On there you can paste your sequence into an entry field (red) or upload a fasta file (blue)
- Afterwards by clicking on Build Model (green) you can start the process

Modelling XVI – SwissModel II

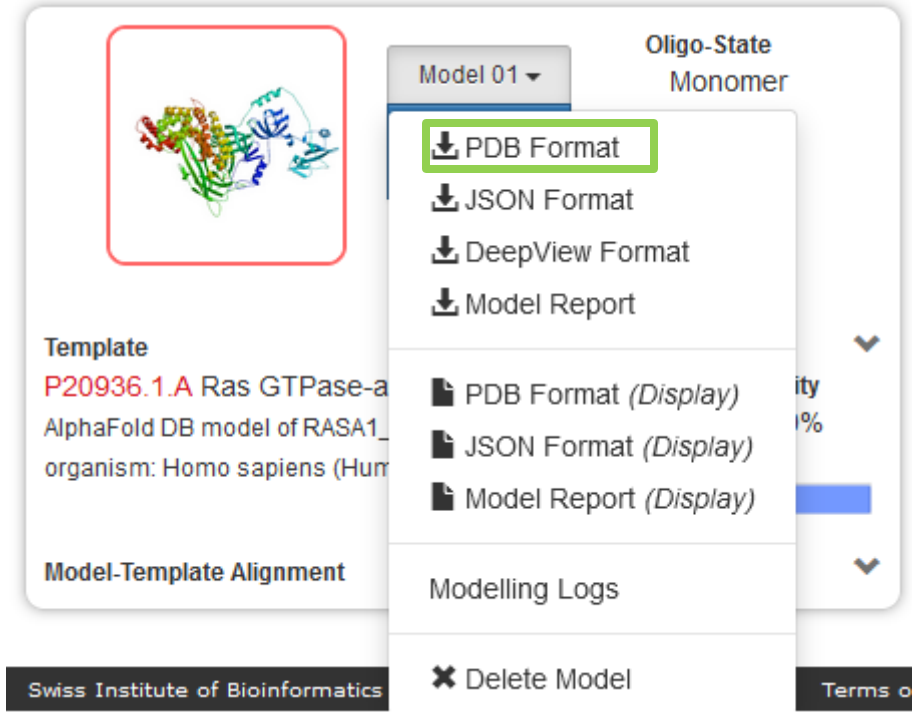
- Once the Modelling job is done you will get 1 or more results as shown below
- These small squares contain information about the used template (green), which parts of your sequence were used (red) and some quality measurements (blue)





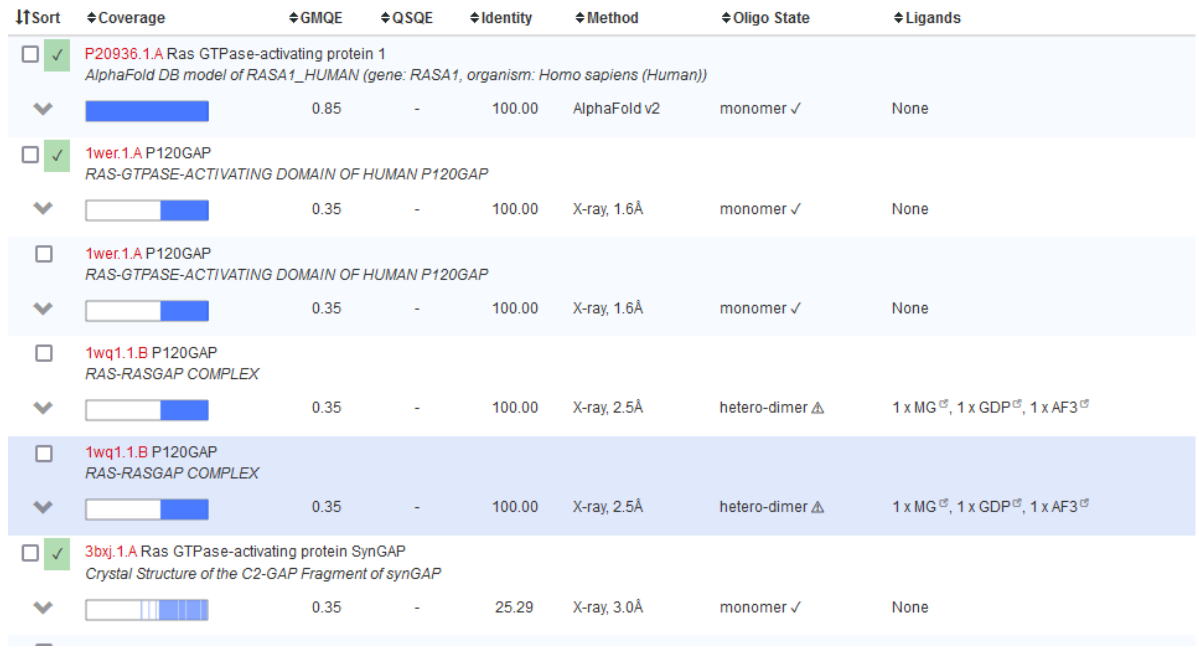
Modelling XVII – SwissModel III

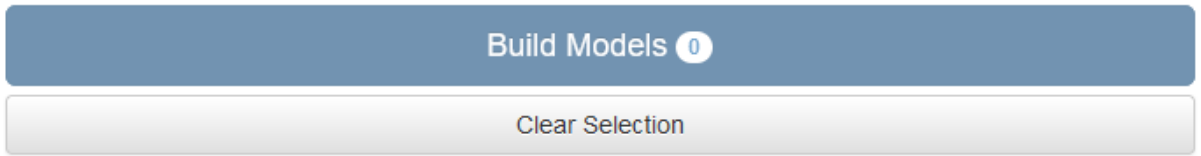
- By clicking on the Name of the generated model, e.g. Model 01, a drop down menu appears in which you can download the coordinate file in the pbd format (green)



Modelling XVIII – SwissModel IV

- If the presented models are not to your liking you can always go to the tab Templates and let SwissModel generate additional models for you using other Templates
- You will see a table in which you can select the templates you want to use (top)
- Afterwards you click on Build Models (bottom) to start a new process





Exercise 08: Modelling with SwissModel

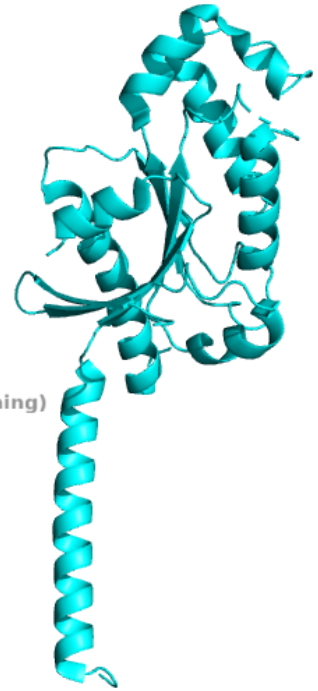
- Use the Mystery Protein Sequences to create some models with the help of the SwissModel Repository (https://swissmodel.expasy.org/interactive)

Modelling XIX – Comparing Structures I

- Once you created a certain amount of models for your sequence you need to compare them, so that you can pick the most optimal one(s) for your further analysis
- Aside from using certain assessment servers you can run a simple comparison using PyMol
- To do so you first have to load in all structures you want to compare into a single Session

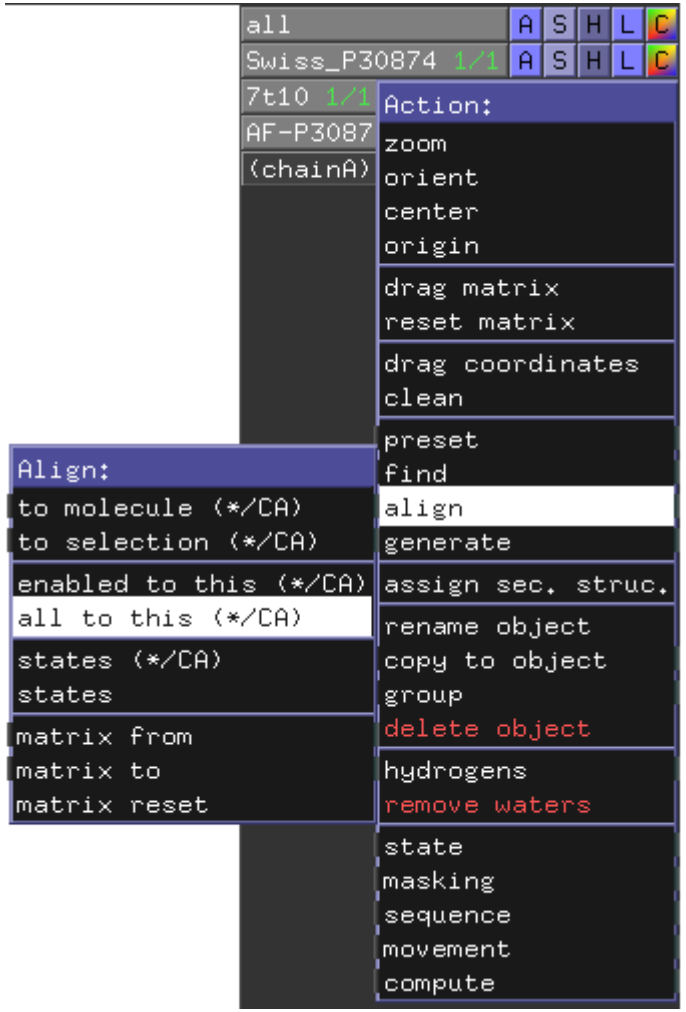
Modelling XX – Comparing Structures II

- After you loaded all structures into your session you will have something as shown on the left



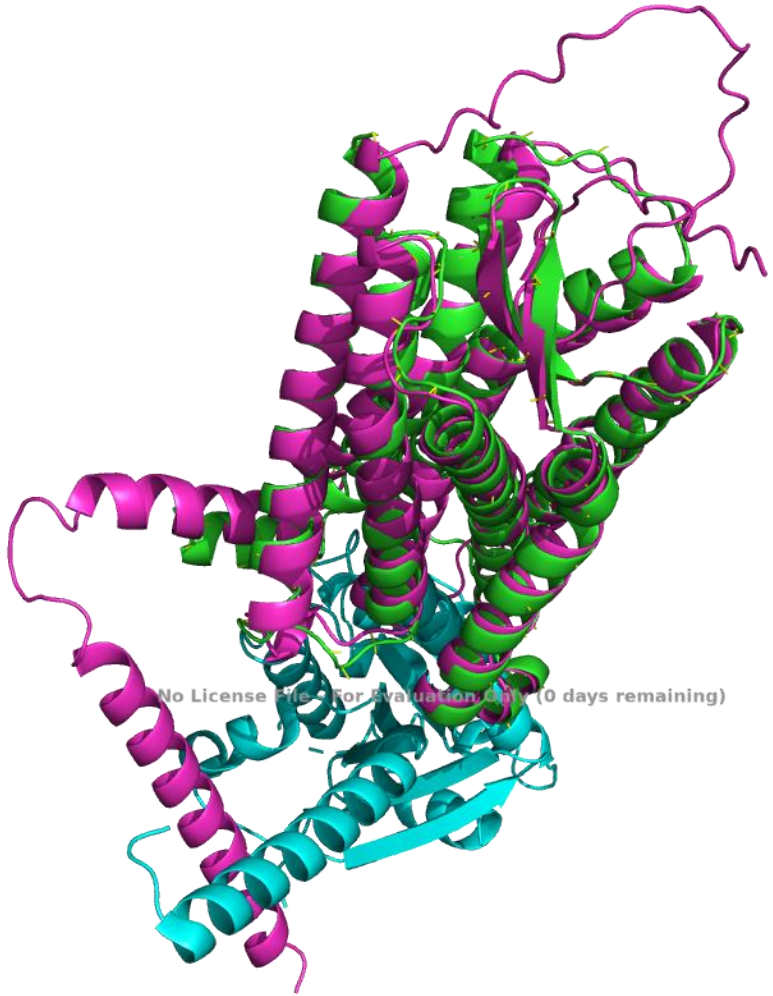
Modelling XXI – Comparing Structures III

- Since the coordinates of your structures probably don‘t match, the structures can be seen all over your session
- So you need to find a way to superpose them so that you can compare them in detail
- To do so, pick one of your structures and use the Action menu
- In there you can find the option align, which contains the function all to this (*/CA) (see image)
- Click on this and all your structures are superposed



Modelling XXII – Comparing Structures IV

- If the structures you superposed are similar enough you will get a nice Bundle of structures as shown on the right
- Now you can compare your structures in more detail by looking at the placement of secondary structures or by comparing how the active centers were modelled



Exercise 09: Comparing structures

- Use at least one of the fasta-files B4DTL8, A0A1C7D1B9 and B3JI28 to create models with the help of the SwissModel repository
- Look up known structures in the UniProt (https://www.uniprot.org/) for these files as well and download them (both PDB and Alphafold versions)
- Compare the structures to each other by aligning them on to each other with the help of PyMol

Biopython.PDB

Biopython.PDB I

- The module biopython contains a large submodule that allows you to deal with pdb-files in Python
- It contains functions for reading in pdb-files as well as several function for calculating bond-lengths, bond-angles and dihedral angles
- It also allows for Nearest-Neighbor searches using kD-Trees
- For more information see:

  https://biopython.org/docs/latest/api/Bio.PDB.html

Biopython.PDB II – Reading Files I

- As we learned during the biopython part, to read in pdb files we need the PDBParser function of the sub module PDB
- Therefore we need to import this sub-sub module first:

```
import Bio.PDB as BP
```

- Then we need to create the parser object:

```
parser = BP.PDBParser()
```

- Afterwards you can read in a pdb-file with the get_structure() method of the parser:

```
structure = parser.get_structure('1kim', '1kim.pdb')
```

Biopython.PDB III – Reading Files II

- When working with files from the RCSB-PDB you could come across warnings when reading in the files:

  PDBConstructionWarning: WARNING: Chain A is discontinuous at line

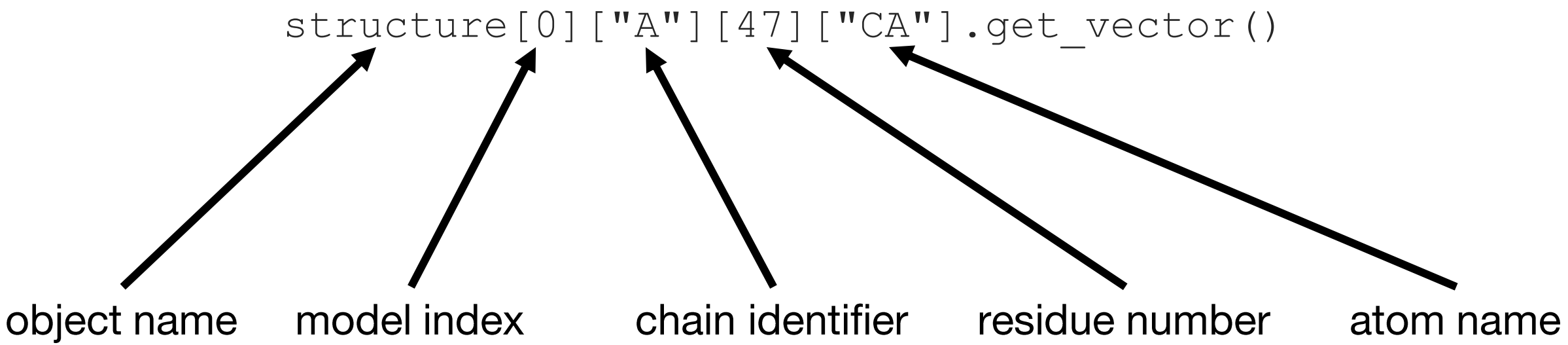
- These warnings tell you potential errors in the corresponding file
- However often times you can simply ignore these warnings

Biopython.PDB IV – Accessing Atoms I

- In most cases when working with pdb-files in Python we are interested in the atoms of our structure
- Biopython allows you to access the coordinates of each atom in a relatively straight forward manner via the get_vector() method
- To get access to the coordinates of the CA-atom of the residue number 47 in the macro-molecule chain A of the first model of our file you could use:

```
structure[0]["A"][47]["CA"].get_vector()
```

Biopython.PDB V – Accessing Atoms II



Exercise 10: Atom Coordinates

- Write a python script that reads in the structure 6ANF
- Print out the coordinates of the atom shown in the table below

Model	Chain	Residue-Number	Atom name
5	A	12	CA
10	A	5	OE1
1	A	9	CB
8	A	10	CZ
2	A	7	HG2
18	A	3	CG

Biopython.PDB VI – Accessing Atoms III

- Another way of accessing different atoms is by looping over the whole structure
- In these cases you need a nested loop construct, that iterates over all models, all macro-molecule chains and all amino acids:

```
for model in structure:
    for chain in model:
        for residue in chain:
```

Biopython.PDB VII – Accessing Atoms IV

- Inside the most inner loop you can than make use of different methods of each residue
- To test which name your current residue has you can use the attribute resname of each amino acid object

```
if residue.resname == "PRO":
```

- To test whether your current amino acid has a certain atom or not you can use the method has_id() of each amino acid object:

```
if residue.has_id("CA"):
```

Exercise 11: Tryptophane atoms

- Write a Python script that reads in the pdb-file 3FQK
- Iterate over the structure and print out the coordinates of the NE1, CZ2 and CH2 atoms of all Tryptophanes (TRP)
- Do not print out any other coordinates

Biopython.PDB VIII – Simple Calculations I

- Once you have access to the coordinates of your atoms you can run some simple calculations with them
- To determine the distance between two atoms, you just need to subtract them from each other:

```
for model in structure:
    for chain in model:
        for residue in chain:
            distance = residue["CA"] - residue["N"]
```

- In the example above I calculate the distance between the CA- and N-atom of each amino acid in a structure

Biopython.PDB IX – Simple Calculations II

- For angles we can use the calc_angle function of the PDB submodule
- In this case we need an additional import:
  - import Bio.PDB as BP
- Then we can use the calc_angle() function with the coordinate vectors of our atoms:

```
BP.calc_angle(residue["N"].get_vector(),residue["CA"].get_vector(),
residue["C"].get_vector())
```

- As you can see you need the vectors of three atoms, to calculate the corresponding angle

Biopython.PDB X – Simple Calculations III

- This function returns the angle in radiant, to get the degree value you need to multiplicate it with 180/π

```
(180/math.pi)*BP.calc_angle(residue["N"].get_vector(),
residue["CA"].get_vector(),residue["C"].get_vector())
```

Biopython.PDB XI – Simple Calculations IV

- Last but not least you‘re able to calculate dihedral angles with the calc_dihedral function:

```
(180/math.pi)*BP.calc_dihedral(residue["CA"].get_vector(),
residue["CB"].get_vector(),residue["CG"].get_vector(),
residue["CD1"].get_vector())
```

- Again the result is in radiant, so you need to convert it to degree
- As you can see you need the vectors of four atoms, to calculate the corresponding dihedral angle

Exercise 12: Structural calculations

- Write a Python script that reads in the structure 7UN2
- The script should calculate the distances, angles and dihedrals described in the table below

Type	Amino acid	Atoms
Distance	ASP	CG – OD1
Distance	ASP	CG – OD2
Distance	HIS	CE1 – ND1
Distance	HIS	CE1 – NE2
Angle	VAL	CA – CB – CG1
Angle	TYR	CE1 – CZ – OH
Dihedral	LYS	CG – CD – CE – NZ
Dihedral	GLN	CB – CG – CD – OE1

Literature I

1. https://sites.cns.utexas.edu/webbgroup/molecular-dynamics-simulations (Last visited: 19.01.2023)
2. https://www.macinchem.org/blog/files/6984e0d3250fad75a74f4f159c1d03f2-1916.php (Last visited: 19.01.2023)
3. https://en.wikipedia.org/wiki/Linus_Pauling (Last edited: 17.12.2022, Last visited: 19.01.2023
4. Pauling, L.; Corey, R.B. (1951): Configuration of polypeptide chains. Nature 4274, 550–551.
5. Pauling, L.; Corey, R.B. (1951): Configurations of Polypeptide Chains With Favored Orientations Around Single Bonds. Two New Pleated Sheets. Proc Natl Acad Sci U S A 11, 729–740.
6. Pauling, L.; Corey, R.B. (1951): The pleated sheet, a new layer configuration of polypeptide chains. Proc Natl Acad Sci U S A 5, 251–256.
7. https://www.historyofinformation.com/detail.php?entryid=4427 (Last edited: 20.12.2022, last visited: 19.01.2023)
8. Pauling, Linus. and Marinacci, Barbara. Linus Pauling : in his own words : selected writings, speeches, and interviews / edited by Barbara Marinacci ; introduction by Linus Pauling Simon & Schuster New York 1995

Literature II

9. https://en.wikipedia.org/wiki/Protein_folding (Last edited: 25.05.2023, Last visited: 25.05.2023)
10. https://commons.wikimedia.org/wiki/File:Alpha_helix.png (Last edited: 27.08.2012, Last visited: 19.01.2023)
11. http://curser.science.ru.nl/content-e/pub_NWI/Bioinformatics%20Summerschool%202006%20light%2003092006/documents/do_7457.htm (Last visited: 20.06.2022)
12. https://en.wikipedia.org/wiki/PyMOL (Last edited: 12.01.2023, Last visited: 19.01.2023)
13. https://pymolwiki.org/index.php/Property_Selectors (Last edited: 29.11.2017, Last visited: 30.05.2023)
14. https://pymolwiki.org/index.php/Main_Page (Last edited: 24.12.2022, Last visited: 19.01.2023)
15. https://pymolwiki.org/index.php/MovieSchool (Last edited: 01.09.2009, Last visited: 19.01.2023)
16. Schwede, T. (2013), Structure 21, 1531-1540
17. Anfinsen, C. B., STUDIES ON THE PRINCIPLES THAT GOVERN THE FOLDING OF PROTEIN CHAINS, Nobel Lecture, 1972, Dec. 11

Literature III

18. Levinthal, Cyrus (1969). "How to Fold Graciously". Mossbauer Spectroscopy in Biological Systems: Proceedings of a Meeting Held at Allerton House, Monticello, Illinois: 22–24. Archived from the original on 2010-10-07.
19. Roy Nassar, Gregory L. Dignon, Rostam M. Razban, Ken A. Dill, The Protein Folding Problem: The Role of Theory, Journal of Molecular Biology, Volume 433, Issue 20, 2021,167126,
20. Callaway E. 'It will change everything': DeepMind's AI makes gigantic leap in solving protein structures. Nature. 2020 Dec;588(7837):203-204. doi: 10.1038/d41586-020-03348-4.
21. https://alphafold.ebi.ac.uk/about (Last visited: 31.05.2023)
22. https://predictioncenter.org/index.cgi (Last visited: 31.05.2023)
23. https://www.icr.ac.uk/blogs/the-drug-discoverer/page-details/reflecting-on-deepmind-s-alphafold-artificial-intelligence-success-what-s-the-real-significance-for-protein-folding-research-and-drug-discovery (Last edited: 21.08.2021, Last visited: 31.05.2023)

Literature IV

24. https://www.deepmind.com/blog/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology (Last edited: 30.11.2020, Last visited: 31.05.2023)